

Basic Graph Algorithms

Graphs

- A graph $G = (V, E)$
 - V = set of vertices
 - E = set of edges = subset of $V \times V$
 - Thus $|E| = O(|V|^2)$

Graph Variations

- Variations:
 - A *connected graph* has a path from every vertex to every other
 - In an *undirected graph*:
 - Edge (u,v) = edge (v,u)
 - No self-loops
 - In a *directed* graph:
 - Edge (u,v) goes from vertex u to vertex v , notated $u \rightarrow v$

Graph Variations

- More variations:
 - A *weighted graph* associates weights with either the edges or the vertices
 - E.g., a road map: edges might be weighted w/ distance
 - A *multigraph* allows multiple edges between the same vertices
 - E.g., the call graph in a program (a function can get called from multiple points in another function)

Graphs

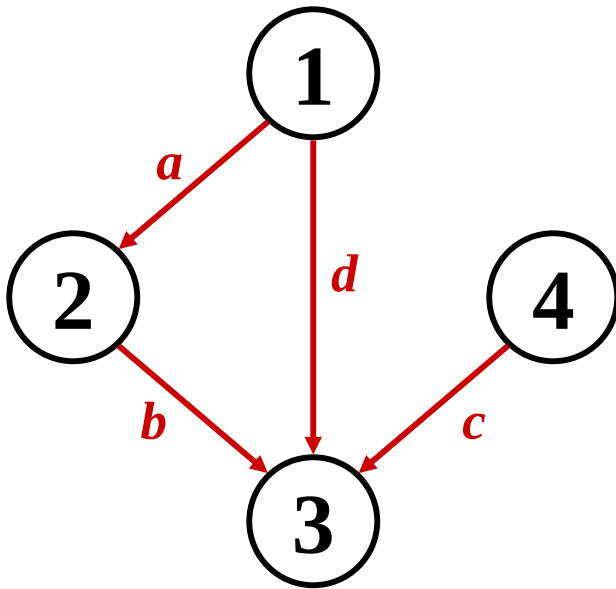
- We will typically express running times in terms of $|E|$ and $|V|$ (often dropping the $|$'s)
 - If $|E| \approx |V|^2$ the graph is *dense*
 - If $|E| \approx |V|$ the graph is *sparse*
- If you know you are dealing with dense or sparse graphs, different data structures may make sense

Representing Graphs

- Assume $V = \{1, 2, \dots, n\}$
- An *adjacency matrix* represents the graph as a $n \times n$ matrix A :
 - $A[i, j] = 1$ if edge $(i, j) \in E$ (or weight of edge)
 $= 0$ if edge $(i, j) \notin E$

Graphs: Adjacency Matrix

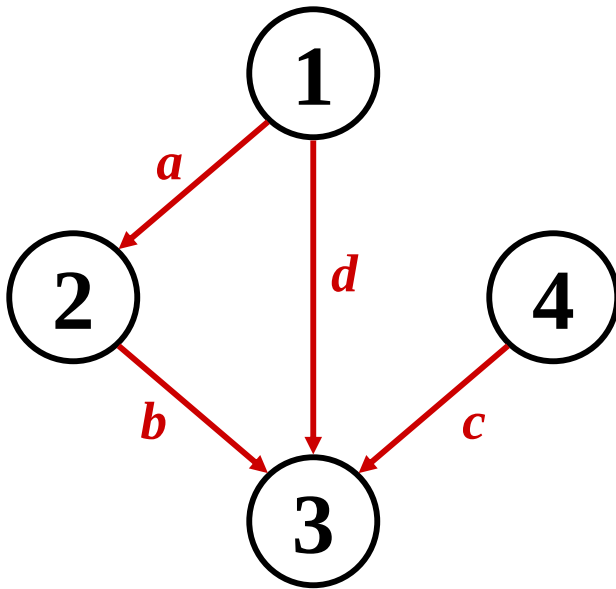
- Example:



A	1	2	3	4
1				
2				
3			??	
4				

Graphs: Adjacency Matrix

- Example:



A	1	2	3	4
1	0	1	1	0
2	0	0	1	0
3	0	0	0	0
4	0	0	1	0

Graphs: Adjacency Matrix

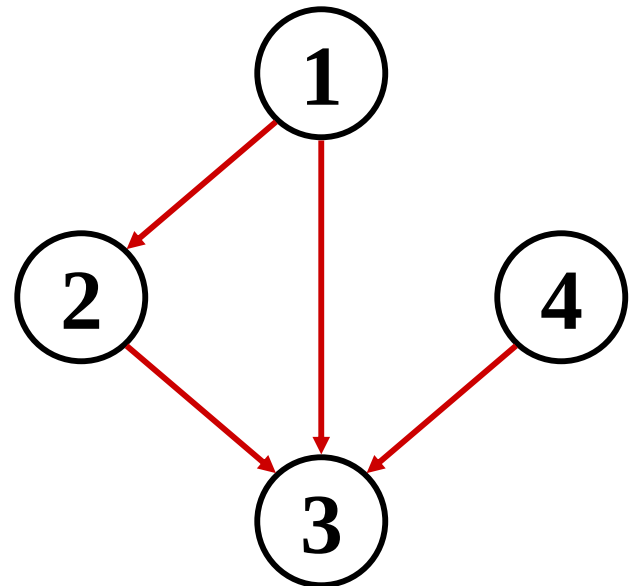
- *How much storage does the adjacency matrix require?*
- A: $O(V^2)$
- *What is the minimum amount of storage needed by an adjacency matrix representation of an undirected graph with 4 vertices?*
- A: 6 bits
 - Undirected graph $\rightarrow$ matrix is symmetric
 - No self-loops $\rightarrow$ don't need diagonal

Graphs: Adjacency Matrix

- The adjacency matrix is a dense representation
 - Usually too much storage for large graphs
 - But can be very efficient for small graphs
- Most large interesting graphs are sparse
 - E.g., planar graphs, in which no edges cross, have $|E| = O(|V|)$ by Euler's formula
 - For this reason the *adjacency list* is often a more appropriate representation

Graphs: Adjacency List

- Adjacency list: for each vertex $v \in V$, store a list of vertices adjacent to v
- Example:
 - $\text{Adj}[1] = \{2,3\}$
 - $\text{Adj}[2] = \{3\}$
 - $\text{Adj}[3] = \{\}$
 - $\text{Adj}[4] = \{3\}$
- Variation: can also keep a list of edges coming *into* vertex



Graphs: Adjacency List

- How much storage is required?
 - The *degree* of a vertex $v = \#$ incident edges
 - Directed graphs have in-degree, out-degree
 - For directed graphs, # of items in adjacency lists is
$$\sum \text{out-degree}(v) = |E|$$
takes $\Theta(V + E)$ storage (*Why?*)
 - For undirected graphs, # items in adj lists is
$$\sum \text{degree}(v) = 2 |E| \quad (\textit{handshaking lemma})$$
also $\Theta(V + E)$ storage
- So: Adjacency lists take $O(V+E)$ storage

Graph Searching

- Given: a graph $G = (V, E)$, directed or undirected
- Goal: methodically explore every vertex and every edge
- Ultimately: build a tree on the graph
 - Pick a vertex as the root
 - Choose certain edges to produce a tree
 - Note: might also build a *forest* if graph is not connected

Breadth-First Search

- “Explore” a graph, turning it into a tree
 - One vertex at a time
 - Expand frontier of explored vertices across the *breadth* of the frontier
- Builds a tree over the graph
 - Pick a *source vertex* to be the root
 - Find (“discover”) its children, then their children, etc.

Breadth-First Search

- Again will associate vertex “colors” to guide the algorithm
 - White vertices have not been discovered
 - All vertices start out white
 - Grey vertices are discovered but not fully explored
 - They may be adjacent to white vertices
 - Black vertices are discovered and fully explored
 - They are adjacent only to black and gray vertices
- Explore vertices by scanning adjacency list of grey vertices

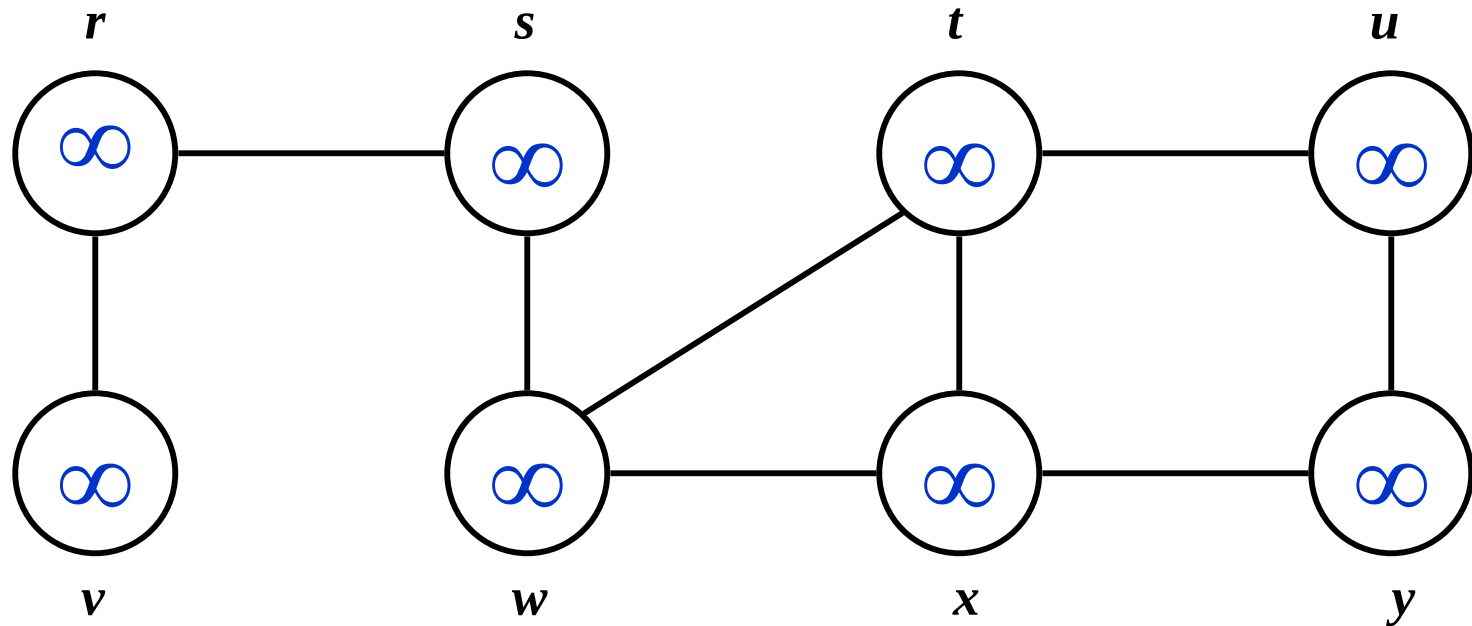
Breadth-First Search

```
BFS(G, s) {  
    initialize vertices;  
    Q = {s};           // Q is a queue (duh); initialize to s  
    while (Q not empty) {  
        u = RemoveTop(Q);  
        for each v ∈ u->adj {  
            if (v->color == WHITE)  
                v->color = GREY;  
                v->d = u->d + 1;  
                v->p = u;  
                Enqueue(Q, v);  
        }  
        u->color = BLACK;  
    }  
}
```

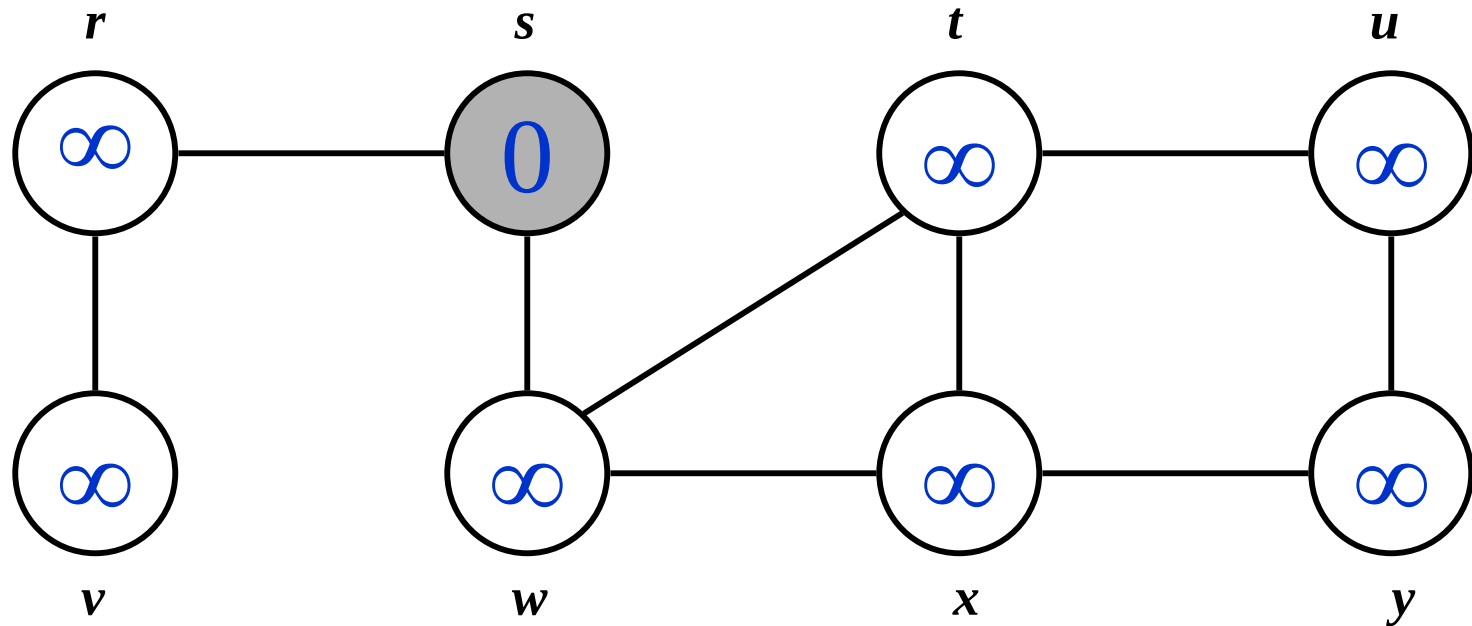
What does v->d represent?

What does v->p represent?

Breadth-First Search: Example

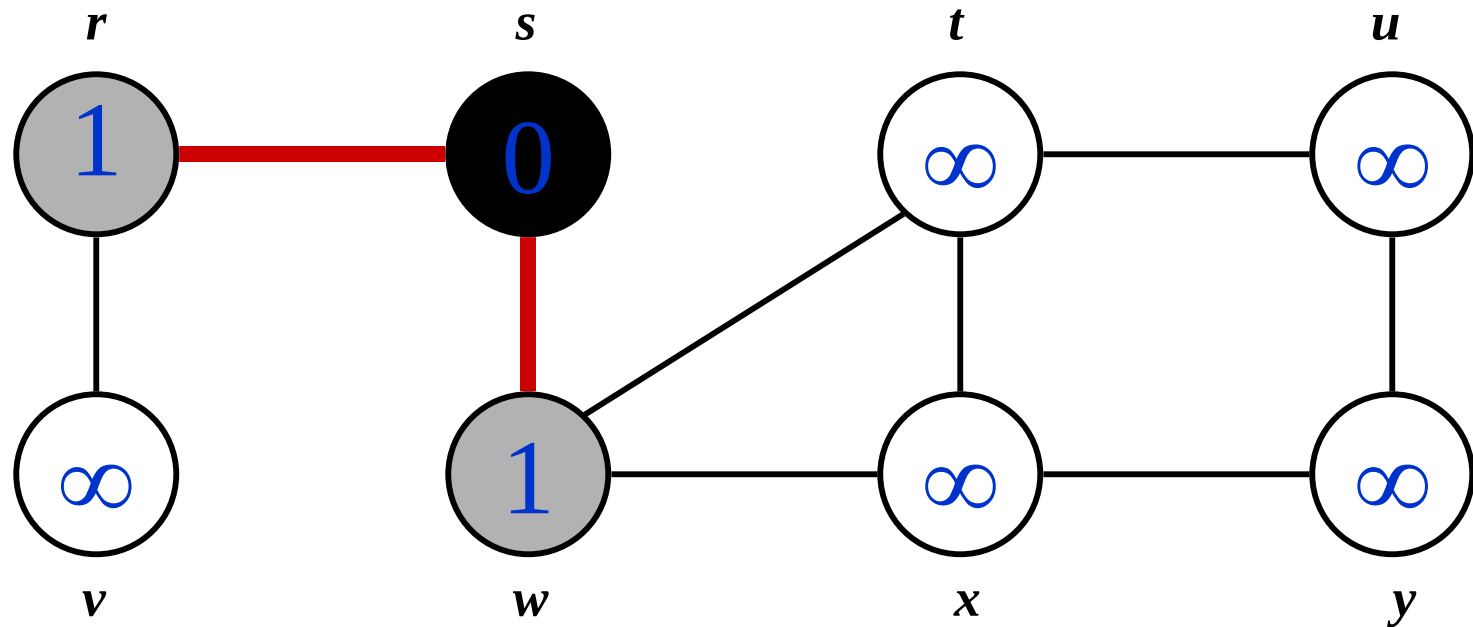


Breadth-First Search: Example



$Q:$ s

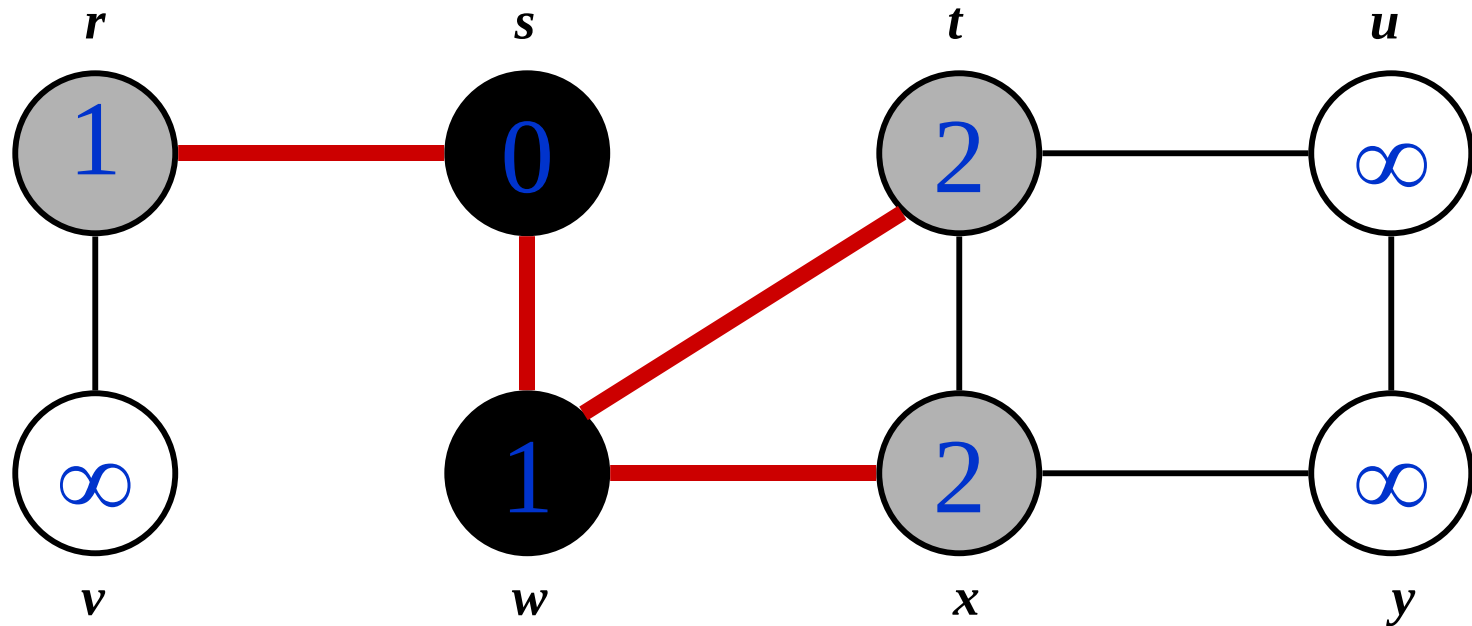
Breadth-First Search: Example



$Q:$

w	r
-----	-----

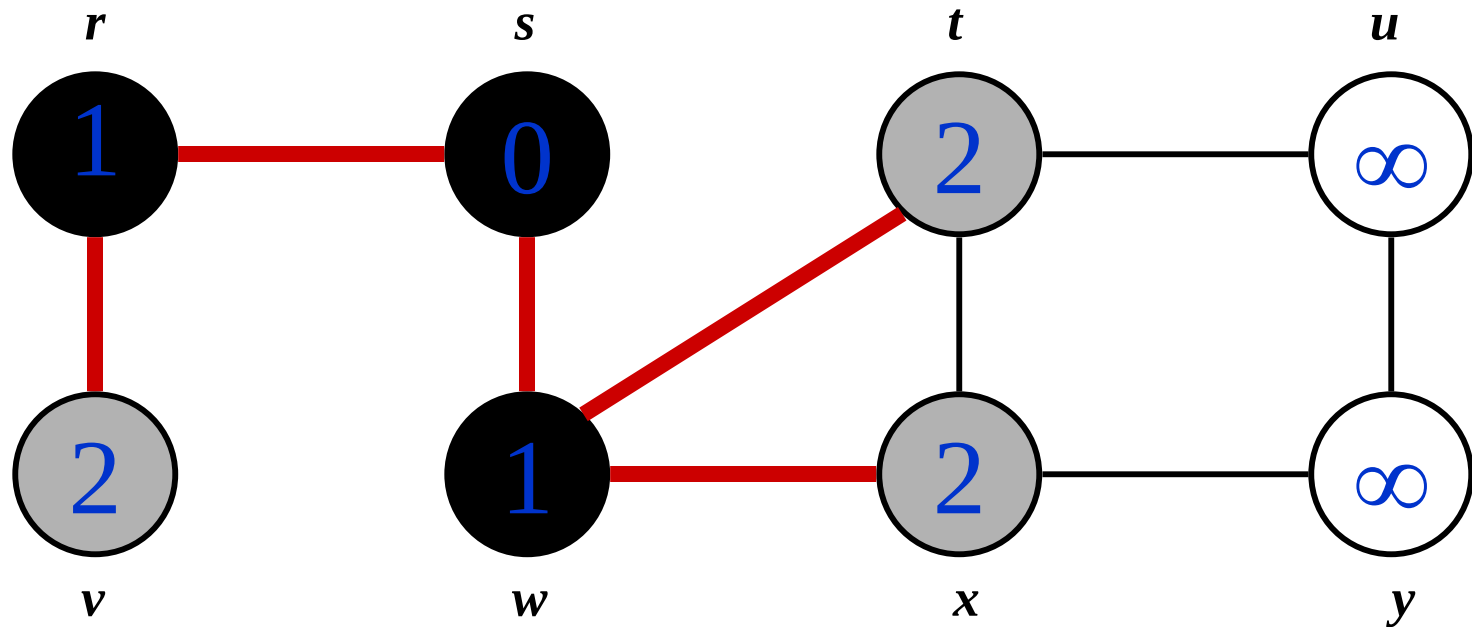
Breadth-First Search: Example



$Q:$

r	t	x
-----	-----	-----

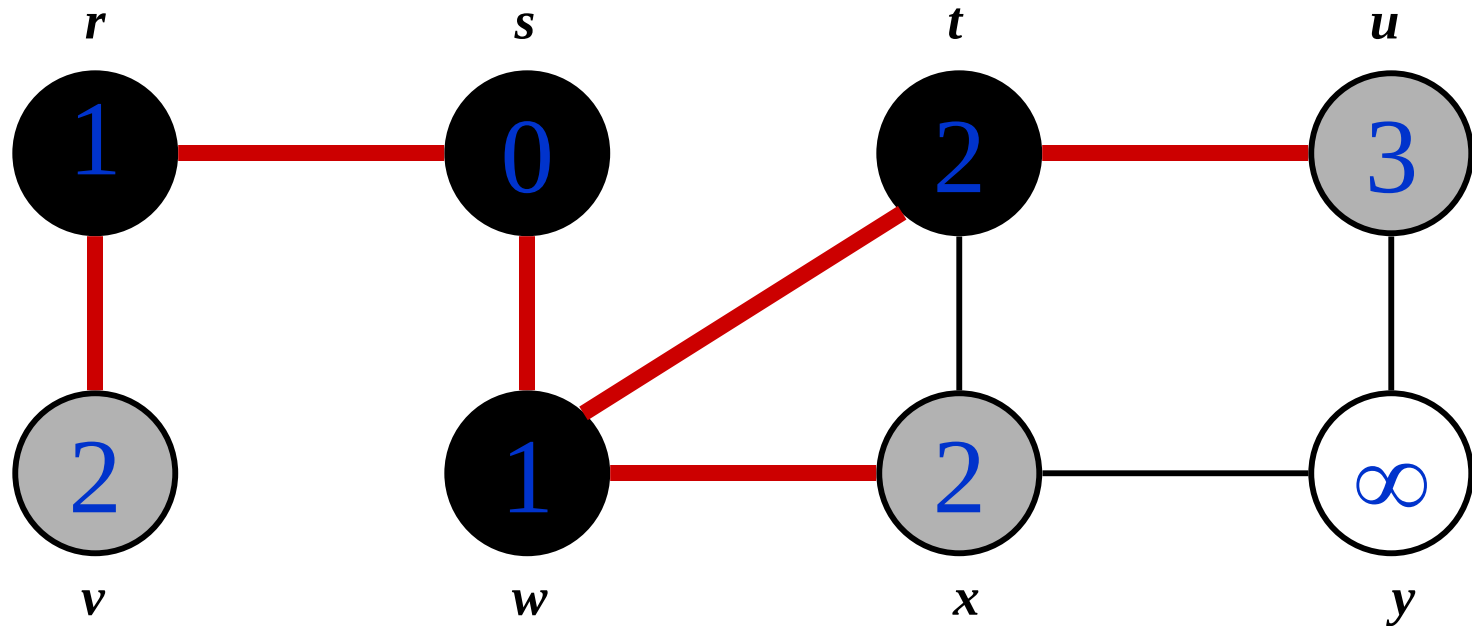
Breadth-First Search: Example



Q :

t	x	v
-----	-----	-----

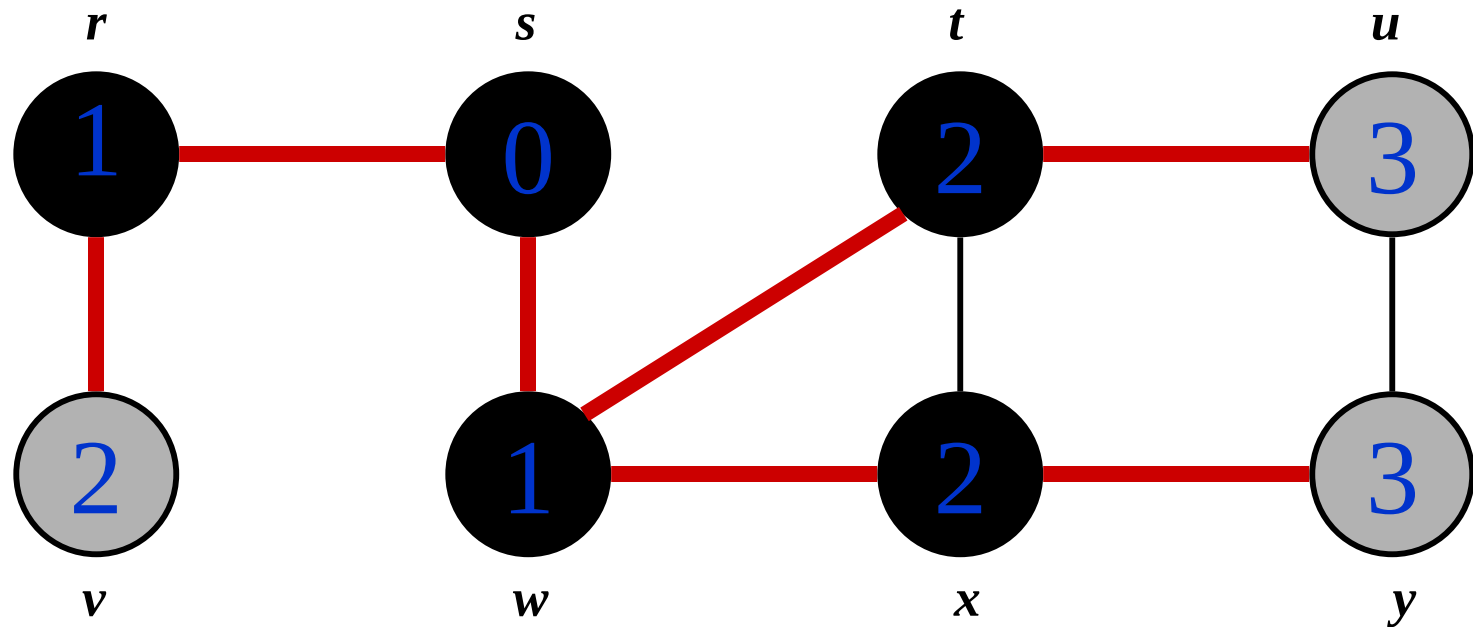
Breadth-First Search: Example



Q :

x	v	u
-----	-----	-----

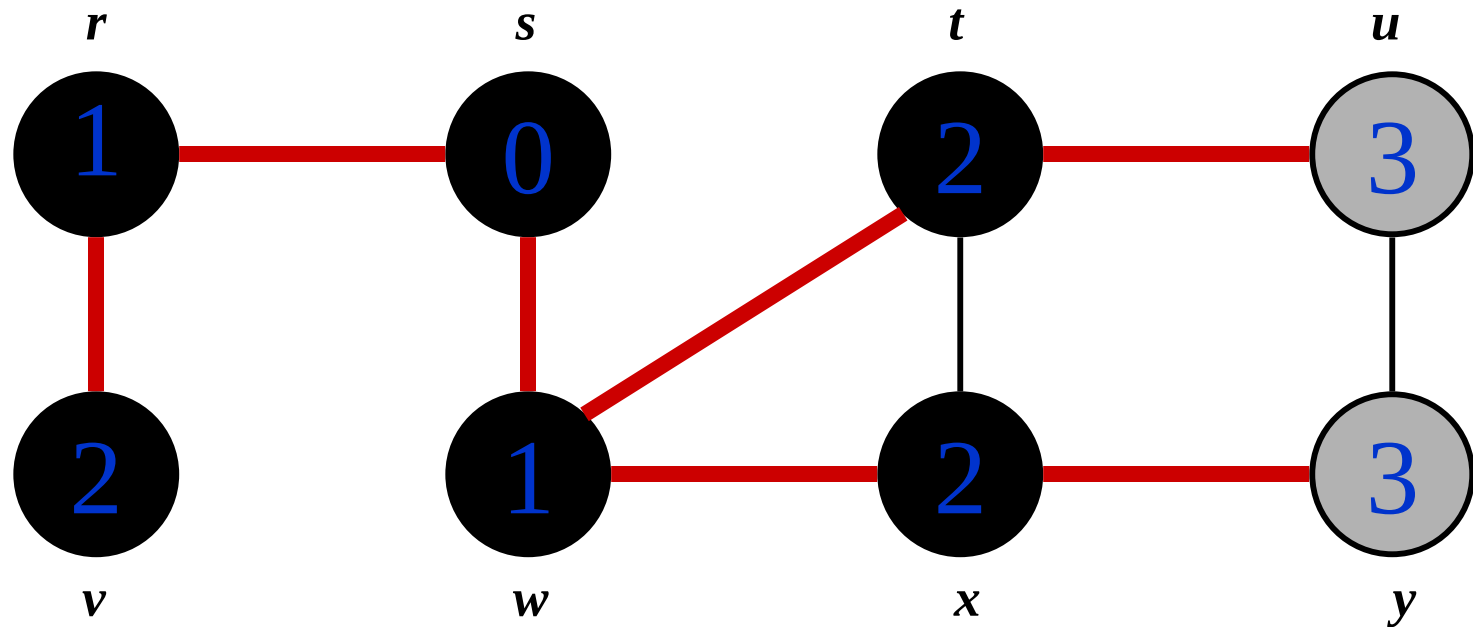
Breadth-First Search: Example



Q :

v	u	y
-----	-----	-----

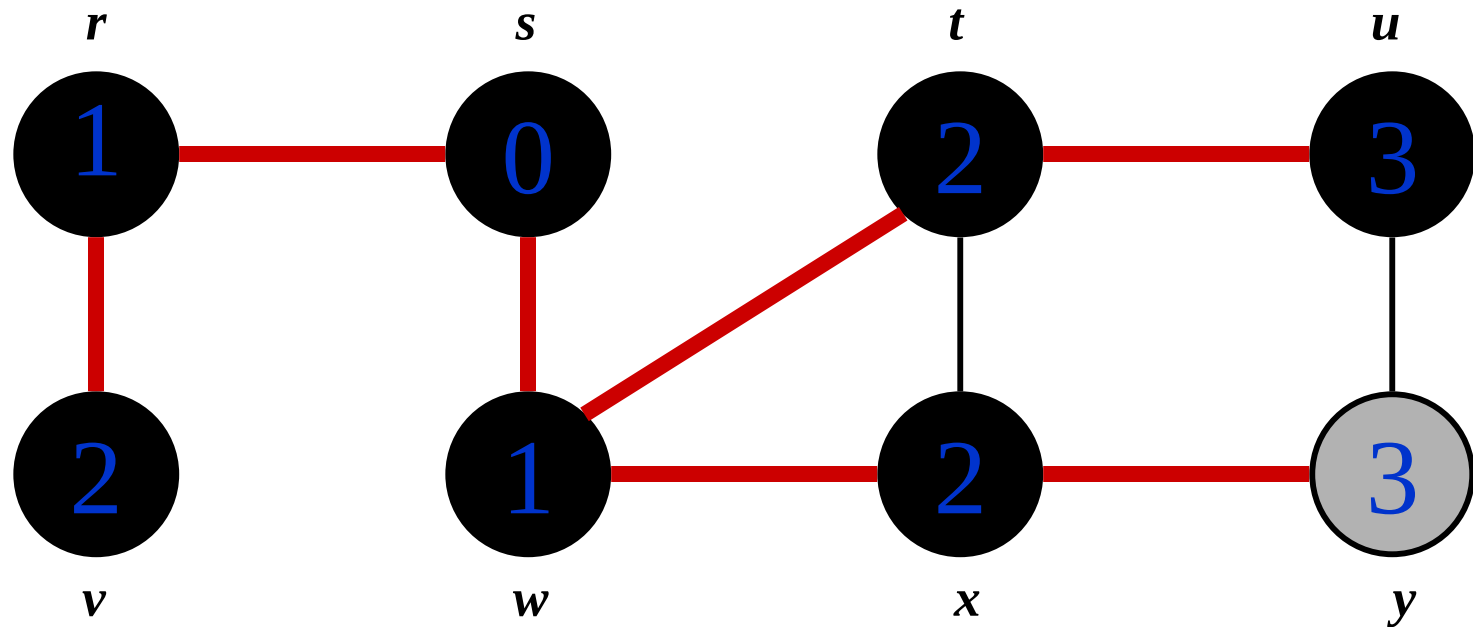
Breadth-First Search: Example



Q :

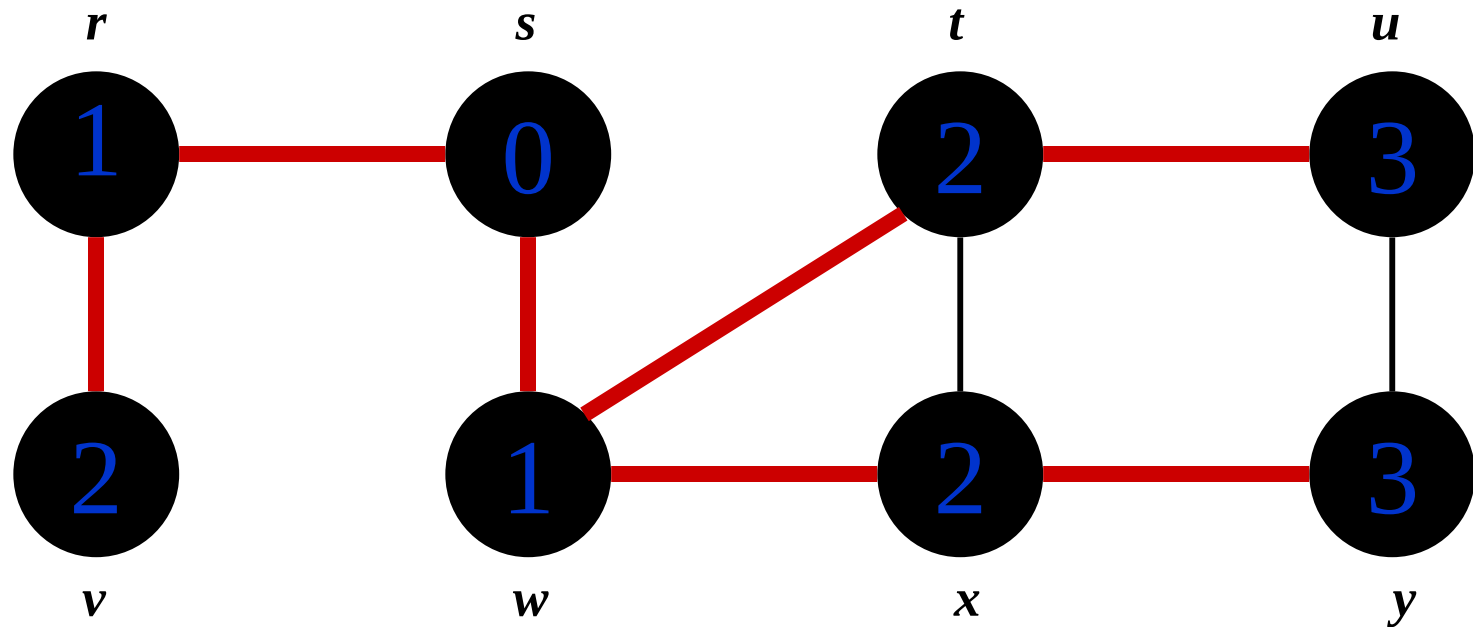
u	y
-----	-----

Breadth-First Search: Example



Q : y

Breadth-First Search: Example



$Q: \emptyset$

BFS: The Code Again

```
BFS(G, s) {  
    initialize vertices;  Touch every vertex:  $O(V)$   
    Q = {s};  
    while (Q not empty) {  
        u = RemoveTop(Q);  u = every vertex, but only once  
        for each v  $\in$  u->adj { (Why?)  
             So v = every vertex that appears in some other vert's adjacency list if (v->color == WHITE)  
                v->color = GREY;  
                v->d = u->d + 1;  
                v->p = u;  
                Enqueue(Q, v);  
        }  
        u->color = BLACK;  
    }  
}
```

What will be the running time?

Total running time: $O(V+E)$

BFS: The Code Again

```
BFS(G, s) {  
    initialize vertices;  
    Q = {s};  
    while (Q not empty) {  
        u = RemoveTop(Q);  
        for each v ∈ u->adj {  
            if (v->color == WHITE)  
                v->color = GREY;  
                v->d = u->d + 1;  
                v->p = u;  
                Enqueue(Q, v);  
        }  
        u->color = BLACK;  
    }  
}
```

*What will be the storage cost
in addition to storing the tree?*

Total space used:

$O(\max(\text{degree}(v))) = O(E)$

Breadth-First Search: Properties

- BFS calculates the *shortest-path distance* to the source node
 - Shortest-path distance $\delta(s,v)$ = minimum number of edges from s to v , or ∞ if v not reachable from s
 - Proof given in the book (p. 472-5)
- BFS builds *breadth-first tree*, in which paths to root represent shortest paths in G
 - Thus can use BFS to calculate shortest path from one vertex to another in $O(V+E)$ time

Depth-First Search

- *Depth-first search* is another strategy for exploring a graph
 - Explore “deeper” in the graph whenever possible
 - Edges are explored out of the most recently discovered vertex v that still has unexplored edges
 - When all of v 's edges have been explored, backtrack to the vertex from which v was discovered

Depth-First Search

- Vertices initially colored white
- Then colored gray when discovered
- Then black when finished

Depth-First Search: The Code

```
DFS (G)
{
    for each vertex  $u \in G \rightarrow V$ 
    {
         $u \rightarrow \text{color} = \text{WHITE};$ 
    }
    time = 0;
    for each vertex  $u \in G \rightarrow V$ 
    {
        if ( $u \rightarrow \text{color} == \text{WHITE}$ )
            DFS_Visit( $u$ );
    }
}
```

```
DFS_Visit( $u$ )
{
     $u \rightarrow \text{color} = \text{GREY};$ 
    time = time+1;
     $u \rightarrow d = \text{time};$ 
    for each  $v \in u \rightarrow \text{Adj}[]$ 
    {
        if ( $v \rightarrow \text{color} == \text{WHITE}$ )
            DFS_Visit( $v$ );
    }
     $u \rightarrow \text{color} = \text{BLACK};$ 
    time = time+1;
     $u \rightarrow f = \text{time};$ 
}
```


Depth-First Search: The Code

```
DFS (G)
{
    for each vertex u ∈ G->V
    {
        u->color = WHITE;
    }
    time = 0;
    for each vertex u ∈ G->V
    {
        if (u->color == WHITE)
            DFS_Visit(u);
    }
}
```

```
DFS_Visit(u)
{
    u->color = GREY;
    time = time+1;
    u->d = time;
    for each v ∈ u->Adj[]
    {
        if (v->color == WHITE)
            DFS_Visit(v);
    }
    u->color = BLACK;
    time = time+1;
    u->f = time;
}
```

What does u->d represent?

Depth-First Search: The Code

```
DFS (G)
{
    for each vertex u ∈ G->V
    {
        u->color = WHITE;
    }
    time = 0;
    for each vertex u ∈ G->V
    {
        if (u->color == WHITE)
            DFS_Visit(u);
    }
}
```

```
DFS_Visit(u)
{
    u->color = GREY;
    time = time+1;
    u->d = time;
    for each v ∈ u->Adj[]
    {
        if (v->color == WHITE)
            DFS_Visit(v);
    }
    u->color = BLACK;
    time = time+1;
    u->f = time;
}
```

What does u->f represent?

Depth-First Search: The Code

```
DFS (G)
{
    for each vertex  $u \in G \rightarrow V$ 
    {
         $u \rightarrow \text{color} = \text{WHITE};$ 
    }
    time = 0;
    for each vertex  $u \in G \rightarrow V$ 
    {
        if ( $u \rightarrow \text{color} == \text{WHITE}$ )
            DFS_Visit( $u$ );
    }
}
```

```
DFS_Visit( $u$ )
{
     $u \rightarrow \text{color} = \text{GREY};$ 
    time = time+1;
     $u \rightarrow d = \text{time};$ 
    for each  $v \in u \rightarrow \text{Adj}[]$ 
    {
        if ( $v \rightarrow \text{color} == \text{WHITE}$ )
            DFS_Visit( $v$ );
    }
     $u \rightarrow \text{color} = \text{BLACK};$ 
    time = time+1;
     $u \rightarrow f = \text{time};$ 
}
```

Will all vertices eventually be colored black?

Depth-First Search: The Code

```
DFS (G)
{
    for each vertex u ∈ G->V
    {
        u->color = WHITE;
    }
    time = 0;
    for each vertex u ∈ G->V
    {
        if (u->color == WHITE)
            DFS_Visit(u);
    }
}
```

```
DFS_Visit(u)
{
    u->color = GREY;
    time = time+1;
    u->d = time;
    for each v ∈ u->Adj[]
    {
        if (v->color == WHITE)
            DFS_Visit(v);
    }
    u->color = BLACK;
    time = time+1;
    u->f = time;
}
```

What will be the running time?

Depth-First Search: The Code

```
DFS (G)
{
    for each vertex u ∈ G->V
    {
        u->color = WHITE;
    }
    time = 0;
    for each vertex u ∈ G->V
    {
        if (u->color == WHITE)
            DFS_Visit(u);
    }
}
```

```
DFS_Visit(u)
{
    u->color = GREY;
    time = time+1;
    u->d = time;
    for each v ∈ u->Adj[]
    {
        if (v->color == WHITE)
            DFS_Visit(v);
    }
    u->color = BLACK;
    time = time+1;
    u->f = time;
}
```

Running time: $O(n^2)$ because call `DFS_Visit` on each vertex, and the loop over `Adj[]` can run as many as $|V|$ times

Depth-First Search: The Code

```
DFS (G)
{
    for each vertex u ∈ G->V
    {
        u->color = WHITE;
    }
    time = 0;
    for each vertex u ∈ G->V
    {
        if (u->color == WHITE)
            DFS_Visit(u);
    }
}
```

```
DFS_Visit(u)
{
    u->color = GREY;
    time = time+1;
    u->d = time;
    for each v ∈ u->Adj[]
    {
        if (v->color == WHITE)
            DFS_Visit(v);
    }
    u->color = BLACK;
    time = time+1;
    u->f = time;
}
```

BUT, there is actually a tighter bound.

How many times will DFS_Visit() actually be called?

Depth-First Search: The Code

```
DFS (G)
{
    for each vertex  $u \in G \rightarrow V$ 
    {
         $u \rightarrow \text{color} = \text{WHITE};$ 
         $u \rightarrow \pi = \text{NIL};$ 
    }
    time = 0;
    for each vertex  $u \in G \rightarrow V$ 
    {
        if ( $u \rightarrow \text{color} == \text{WHITE}$ )
            DFS_Visit( $u$ );
    }
}
```

```
DFS_Visit( $u$ )
{
     $u \rightarrow \text{color} = \text{GREY};$ 
    time = time+1;
     $u \rightarrow d = \text{time};$ 
    for each  $v \in u \rightarrow \text{Adj}[]$ 
    {
        if ( $v \rightarrow \text{color} == \text{WHITE}$ )
             $v \rightarrow \pi = u;$ 
            DFS_Visit( $v$ );
    }
     $u \rightarrow \text{color} = \text{BLACK};$ 
    time = time+1;
     $u \rightarrow f = \text{time};$ 
}
```

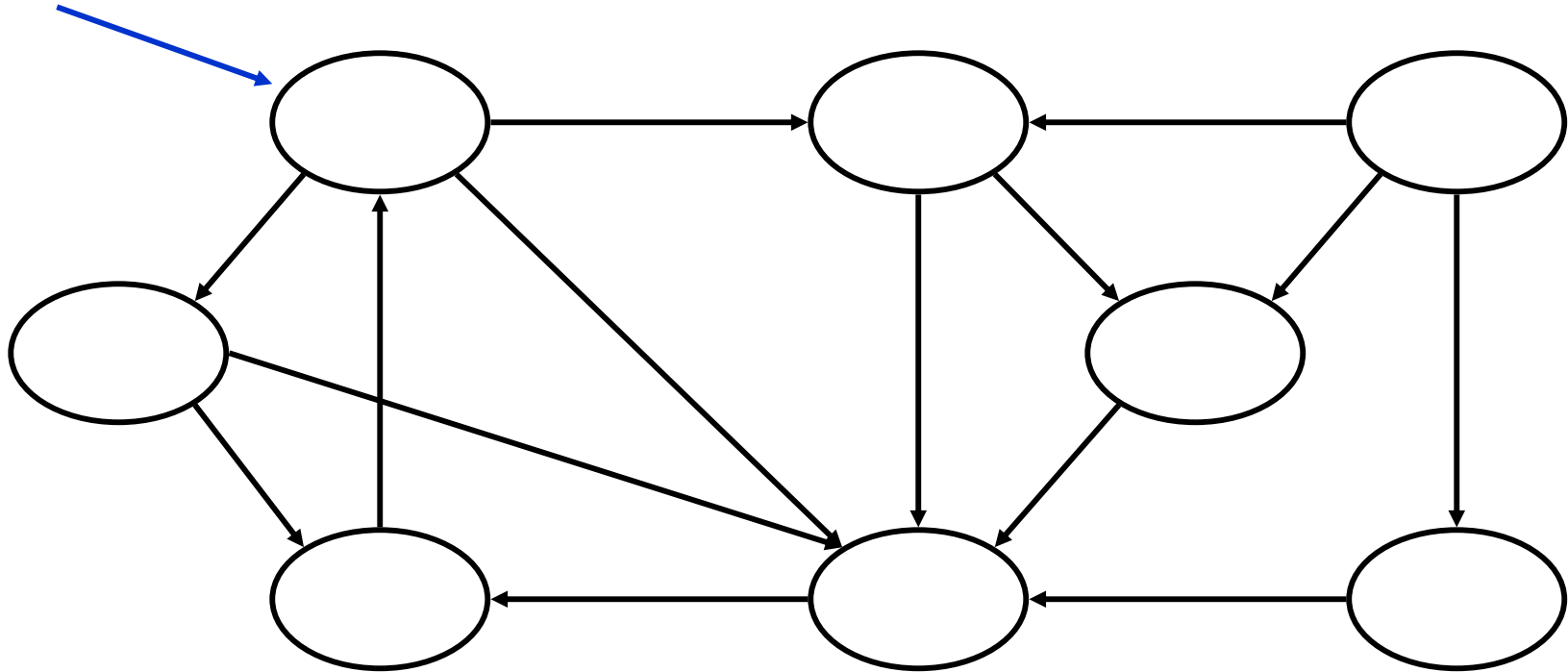
So, running time of DFS = $O(V+E)$

Depth-First Sort Analysis

- This running time argument is an informal example of *amortized analysis* 平摊分析
 - “Charge” the exploration of edge to the edge:
 - Each loop in DFS_Visit can be attributed to an edge in the graph
 - Runs once/edge if directed graph, twice if undirected
 - Thus loop will run in $O(E)$ time, algorithm $O(V+E)$
 - ◆ Considered linear for graph, b/c adj list requires $O(V+E)$ storage
 - Important to be comfortable with this kind of reasoning and analysis

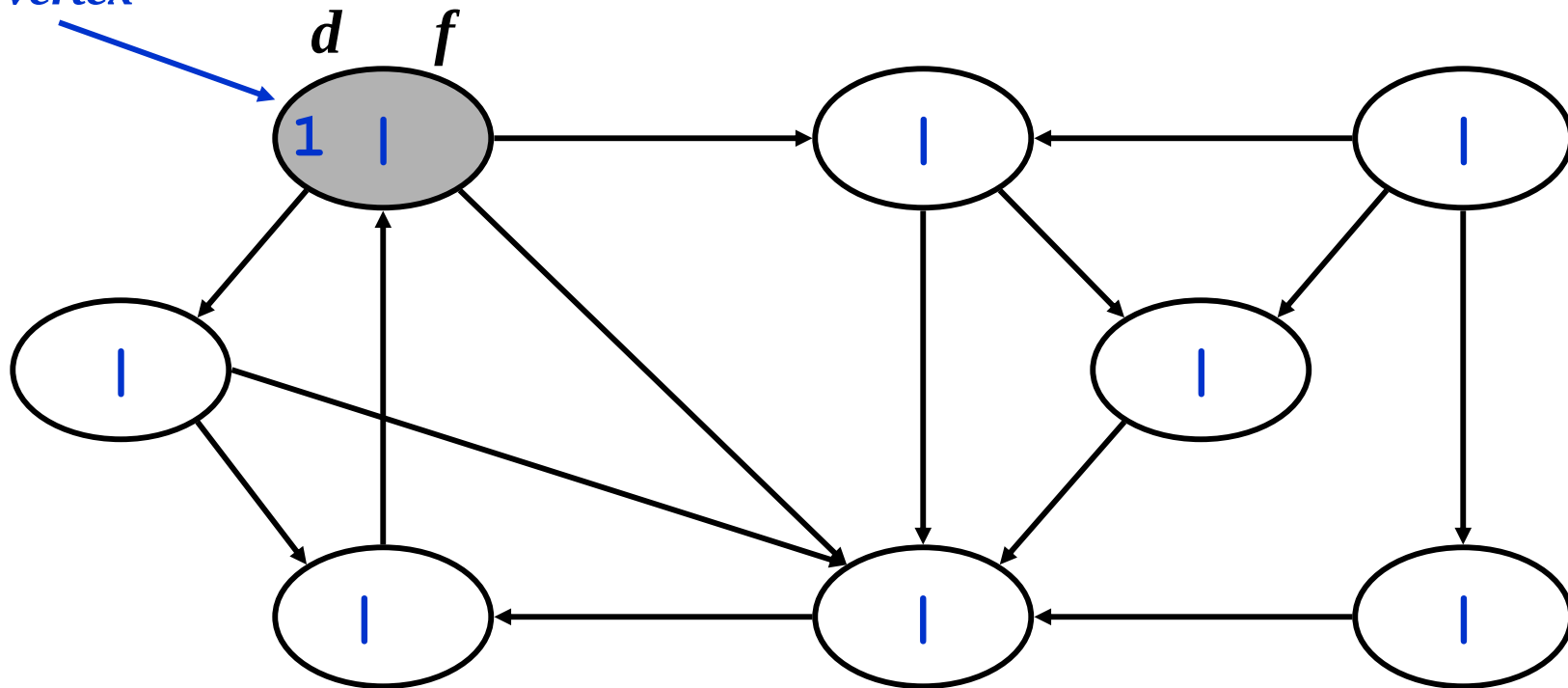
DFS Example

*source
vertex*

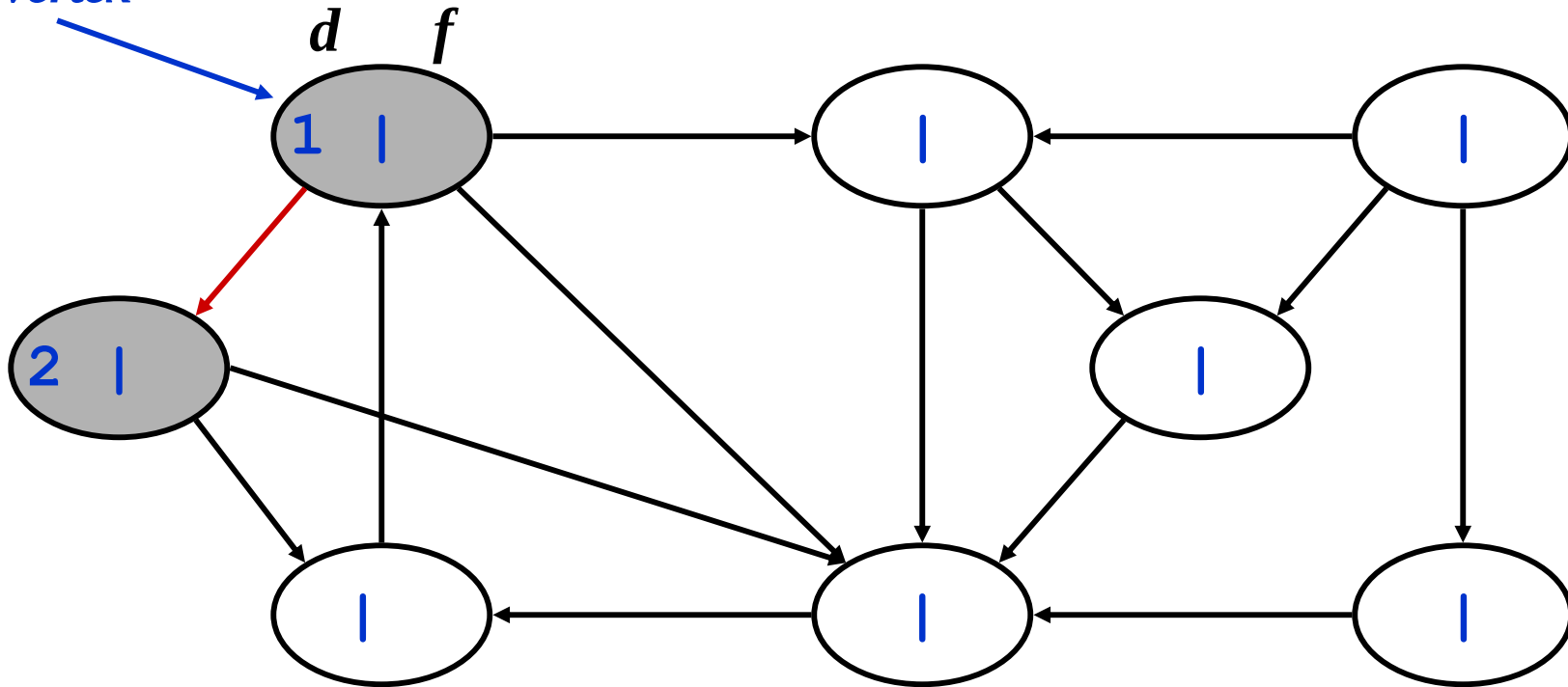


DFS Example

source
vertex

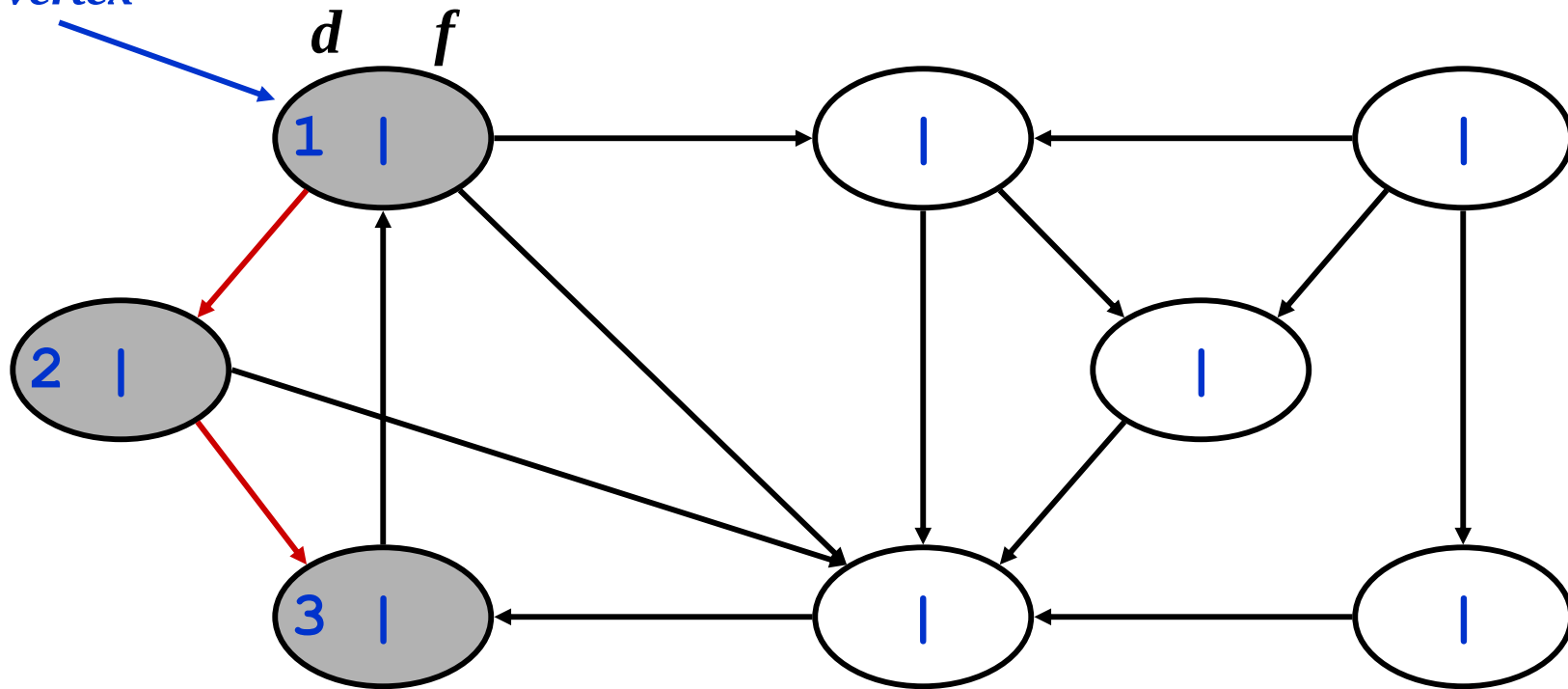


DFS Example



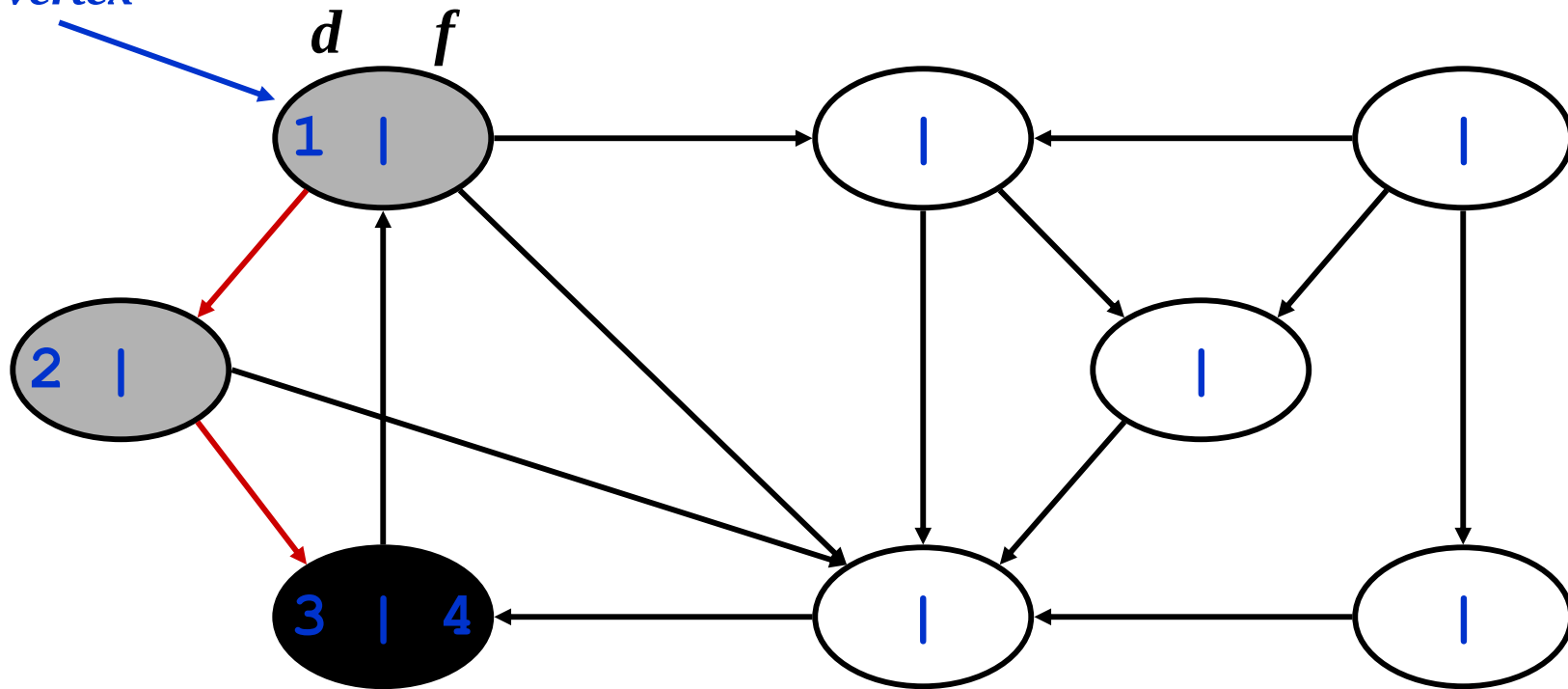
DFS Example

source
vertex



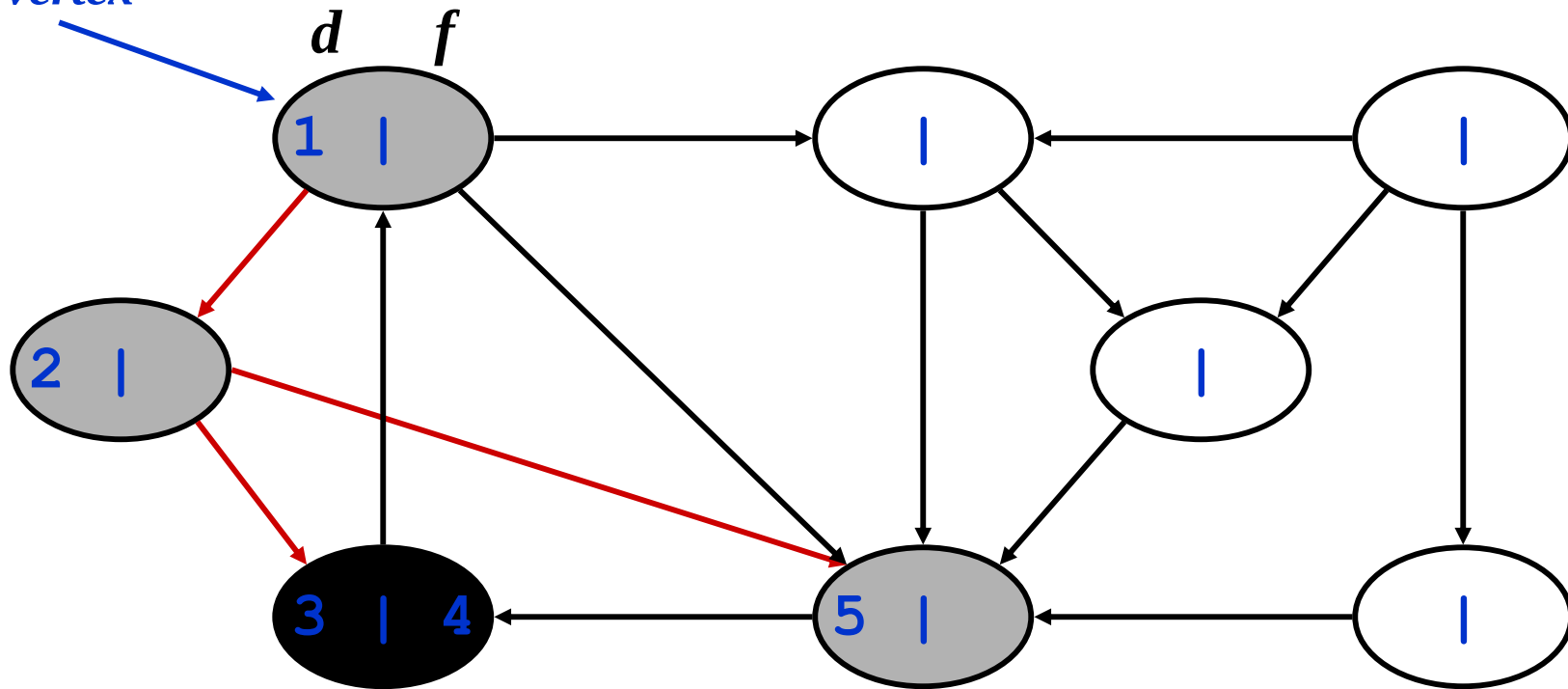
DFS Example

source
vertex



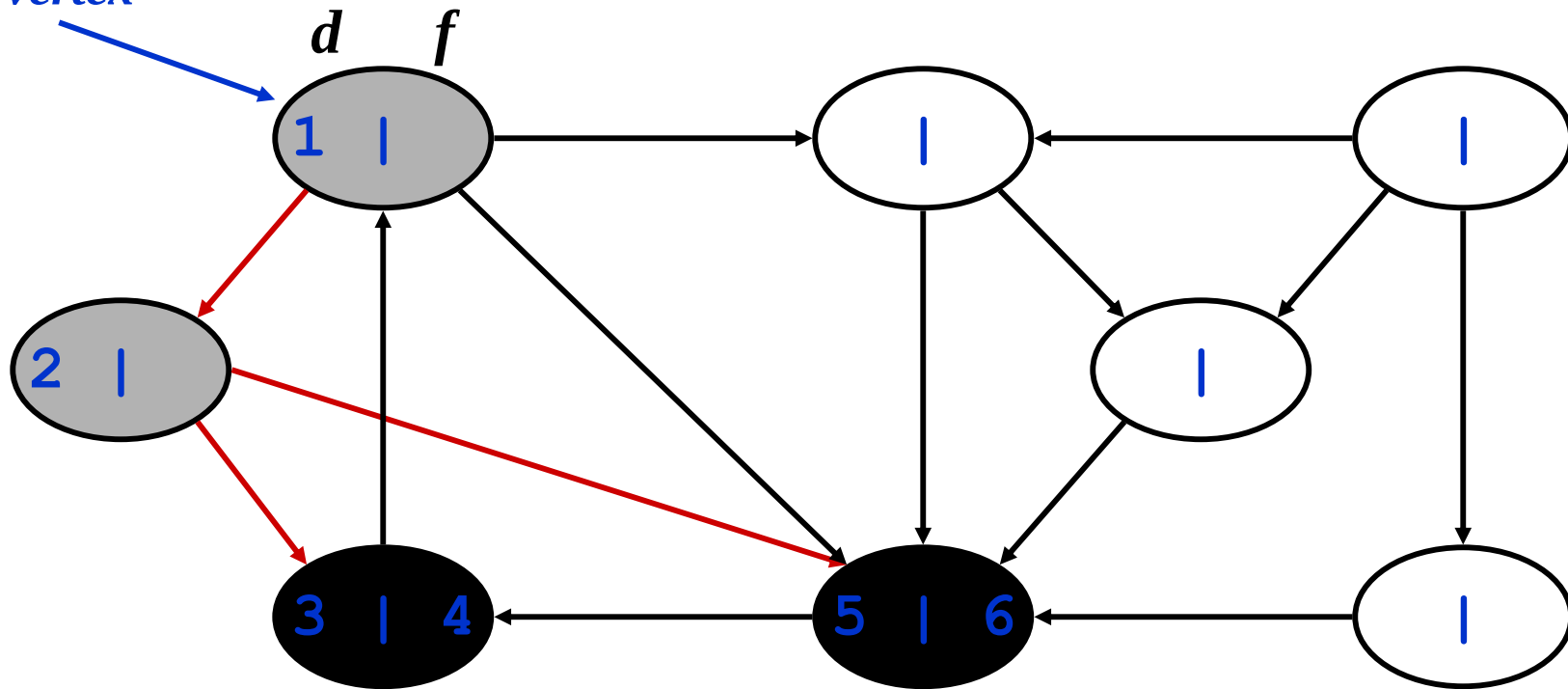
DFS Example

source
vertex



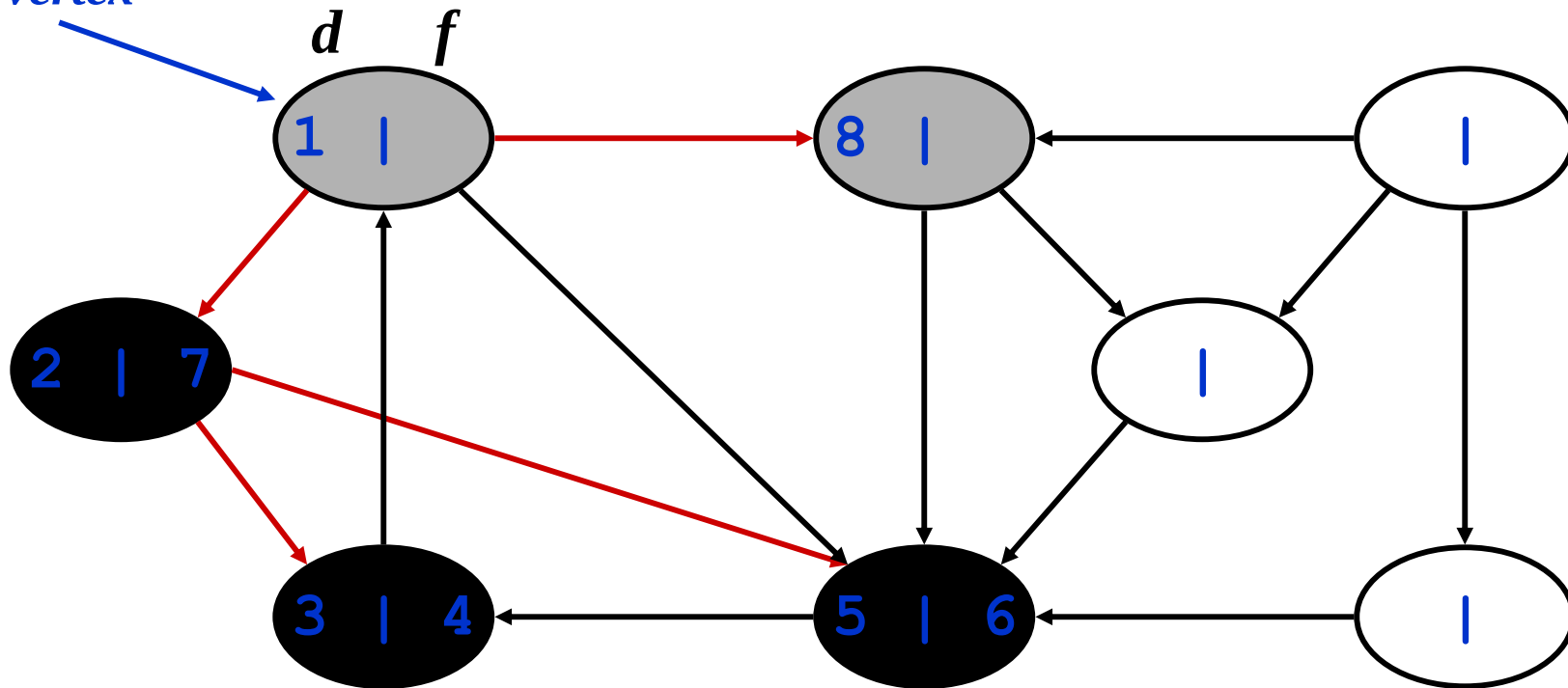
DFS Example

source
vertex



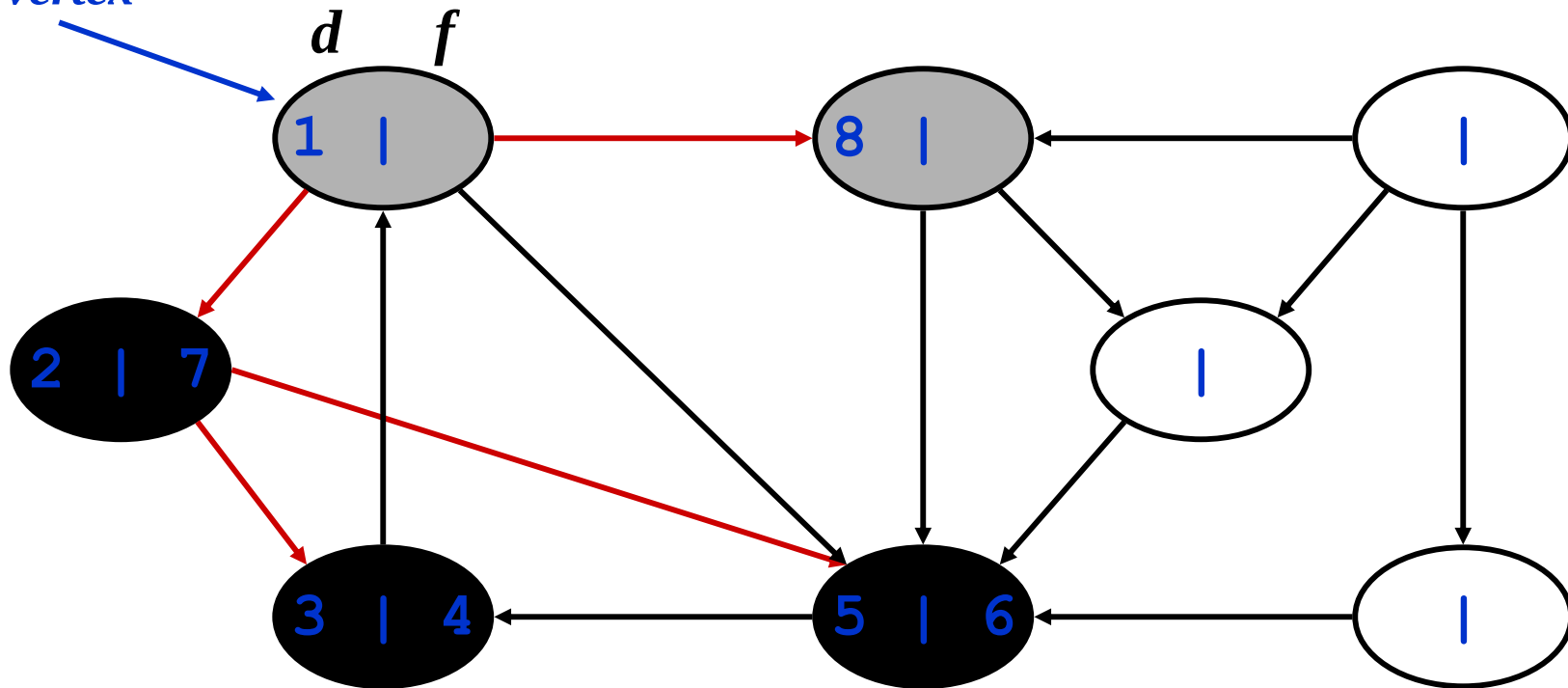
DFS Example

source
vertex



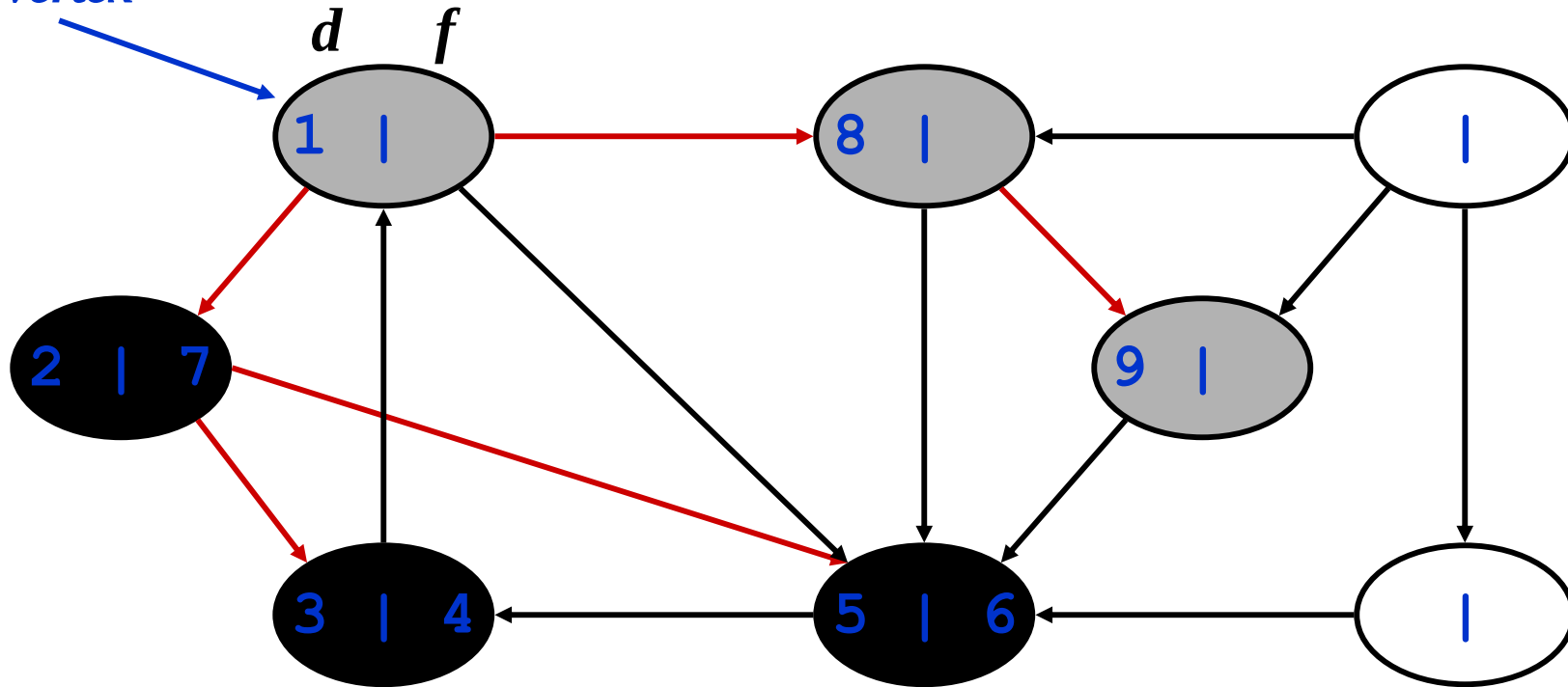
DFS Example

source
vertex



DFS Example

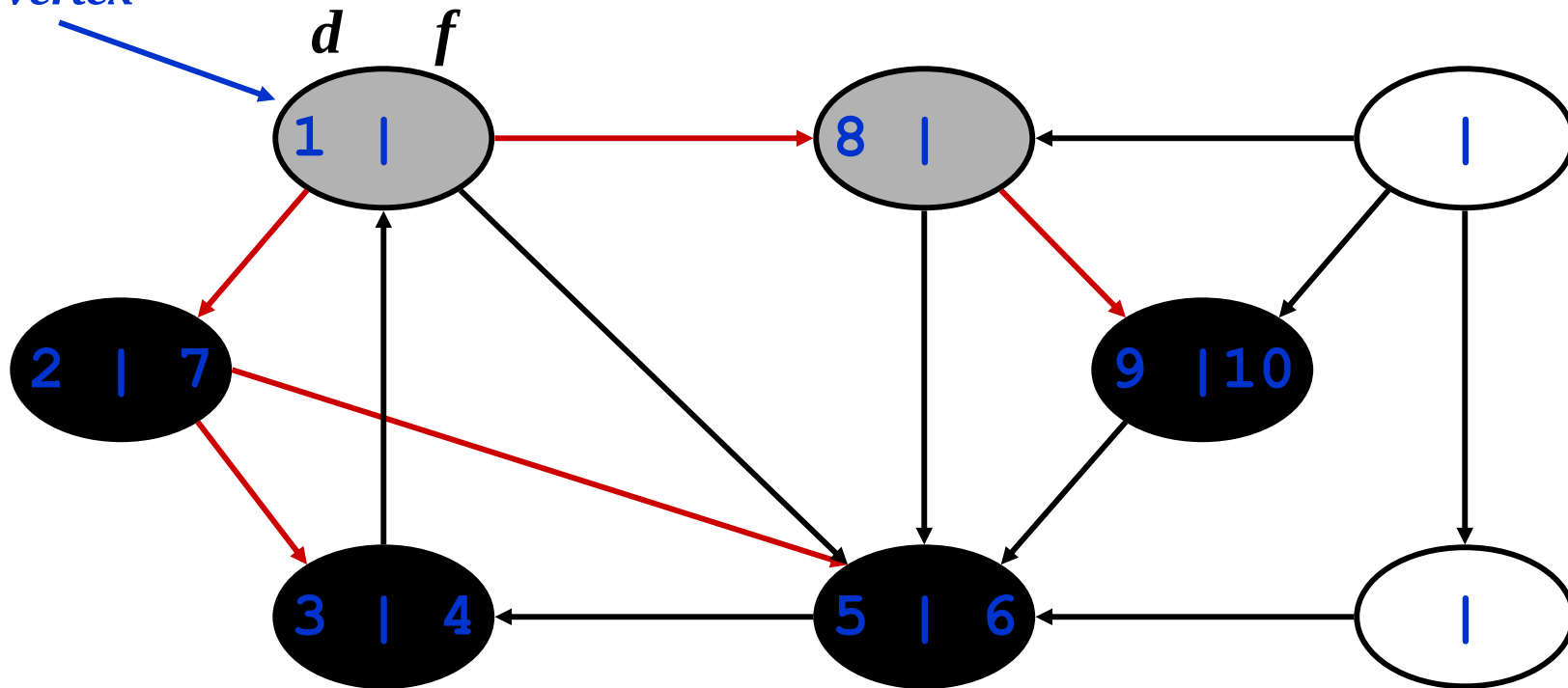
*source
vertex*



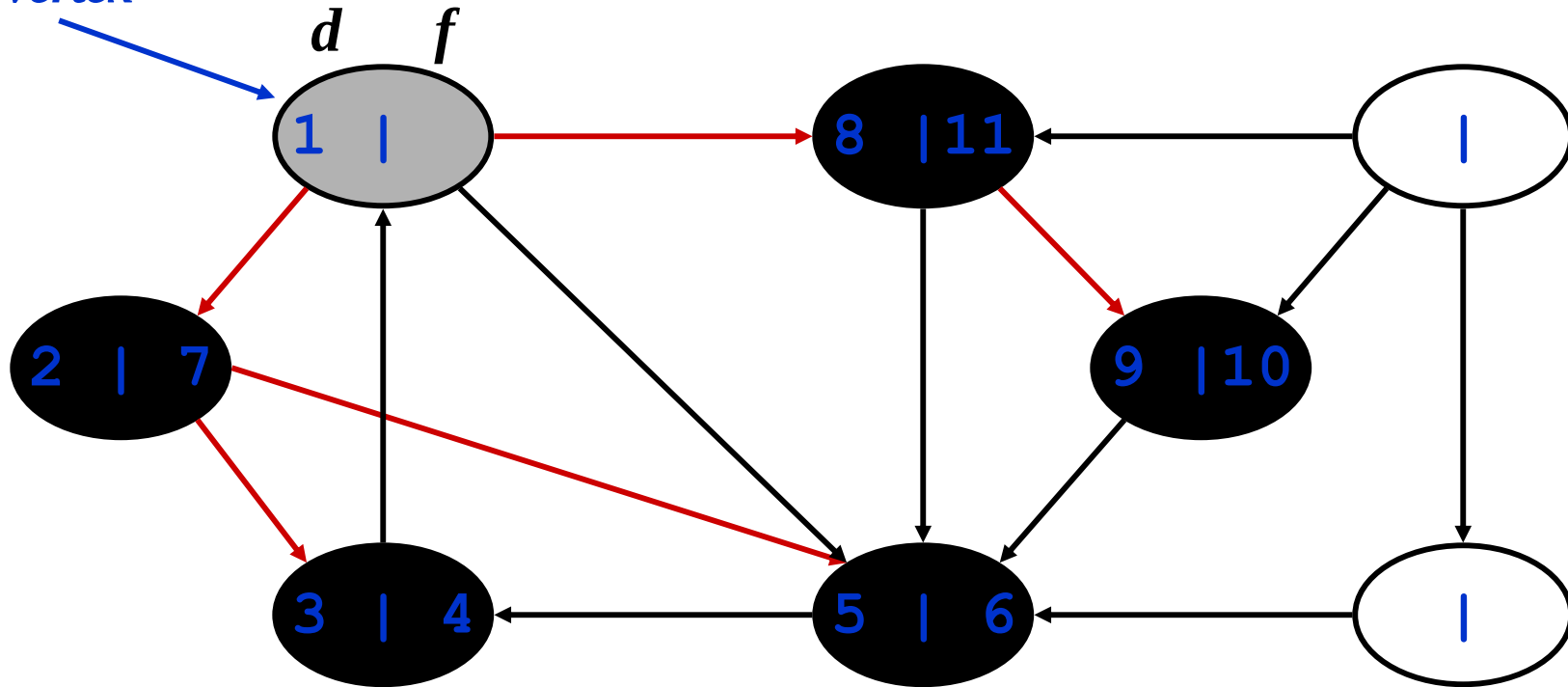
*What is the structure of the grey vertices?
What do they represent?*

DFS Example

source
vertex

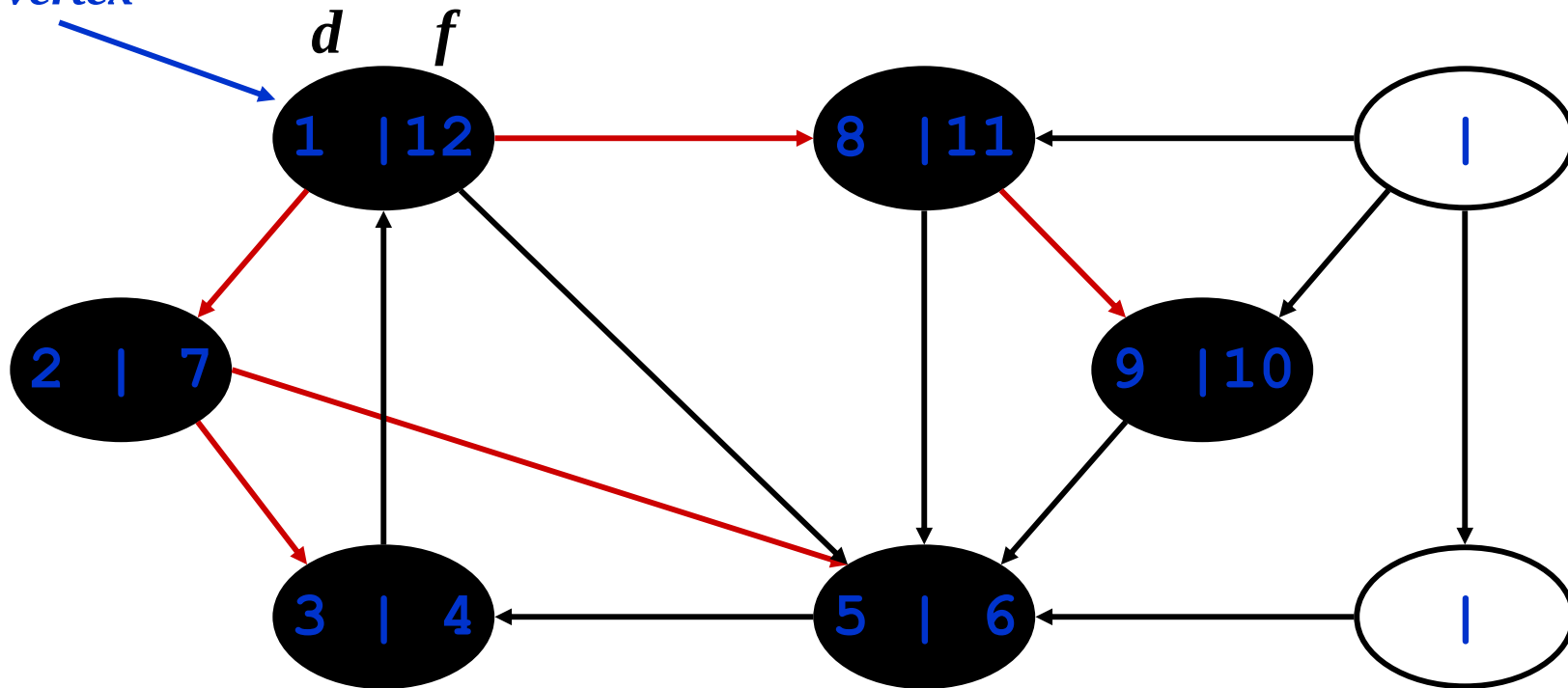


DFS Example



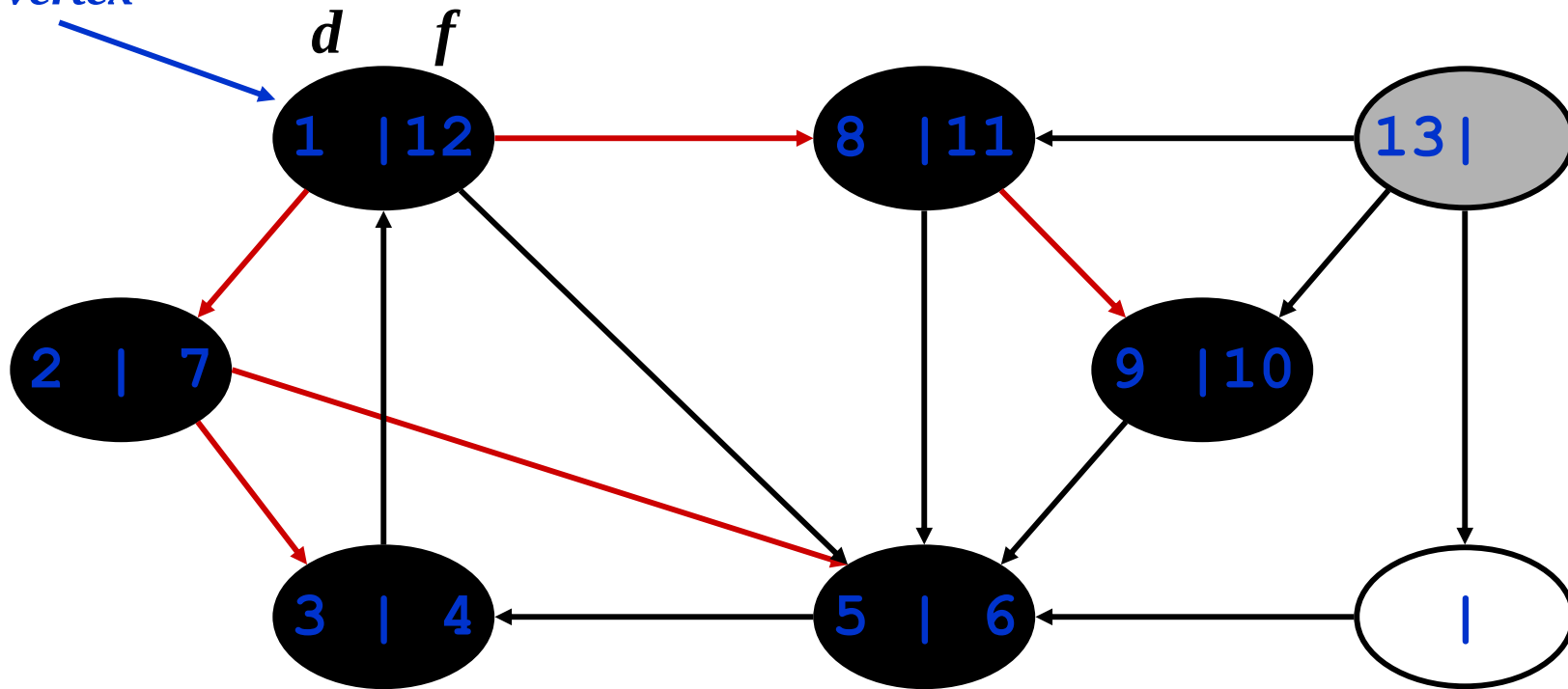
DFS Example

source
vertex



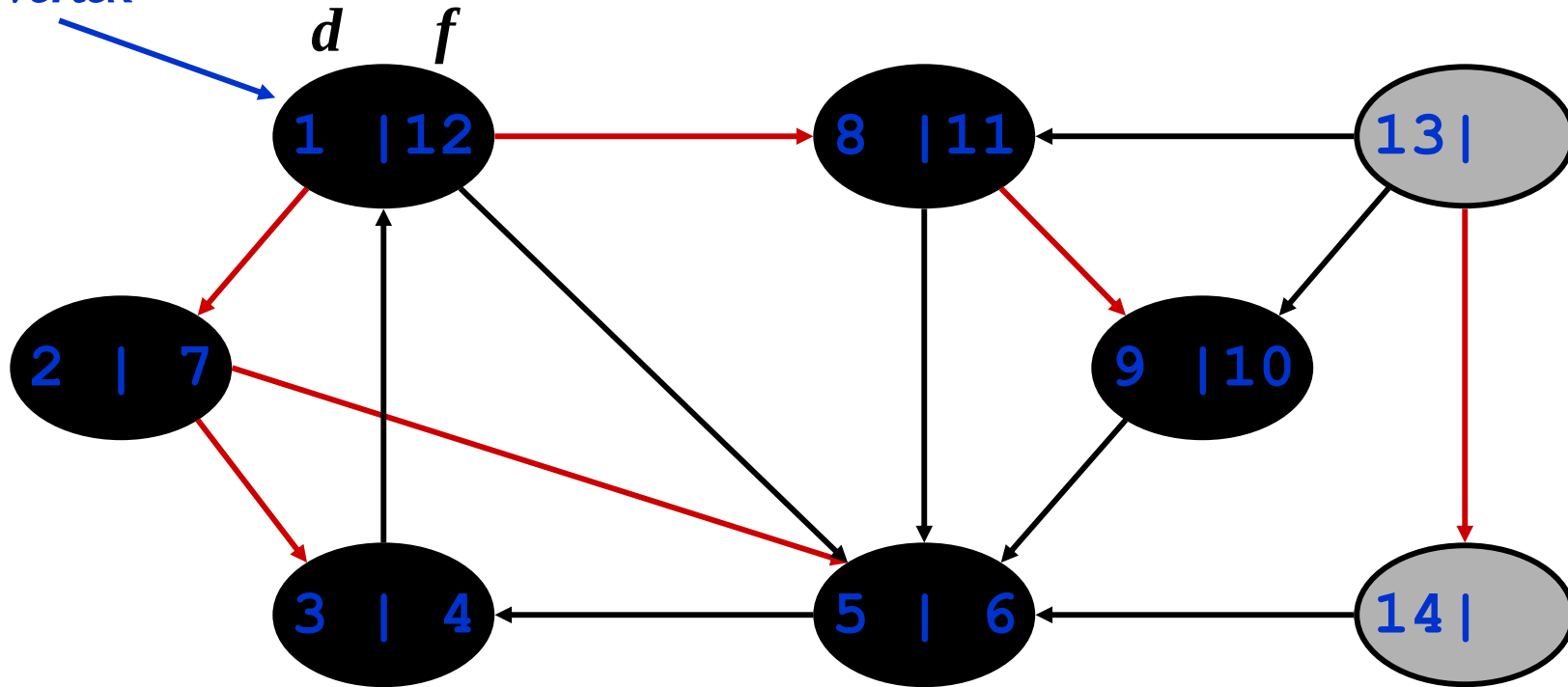
DFS Example

source
vertex



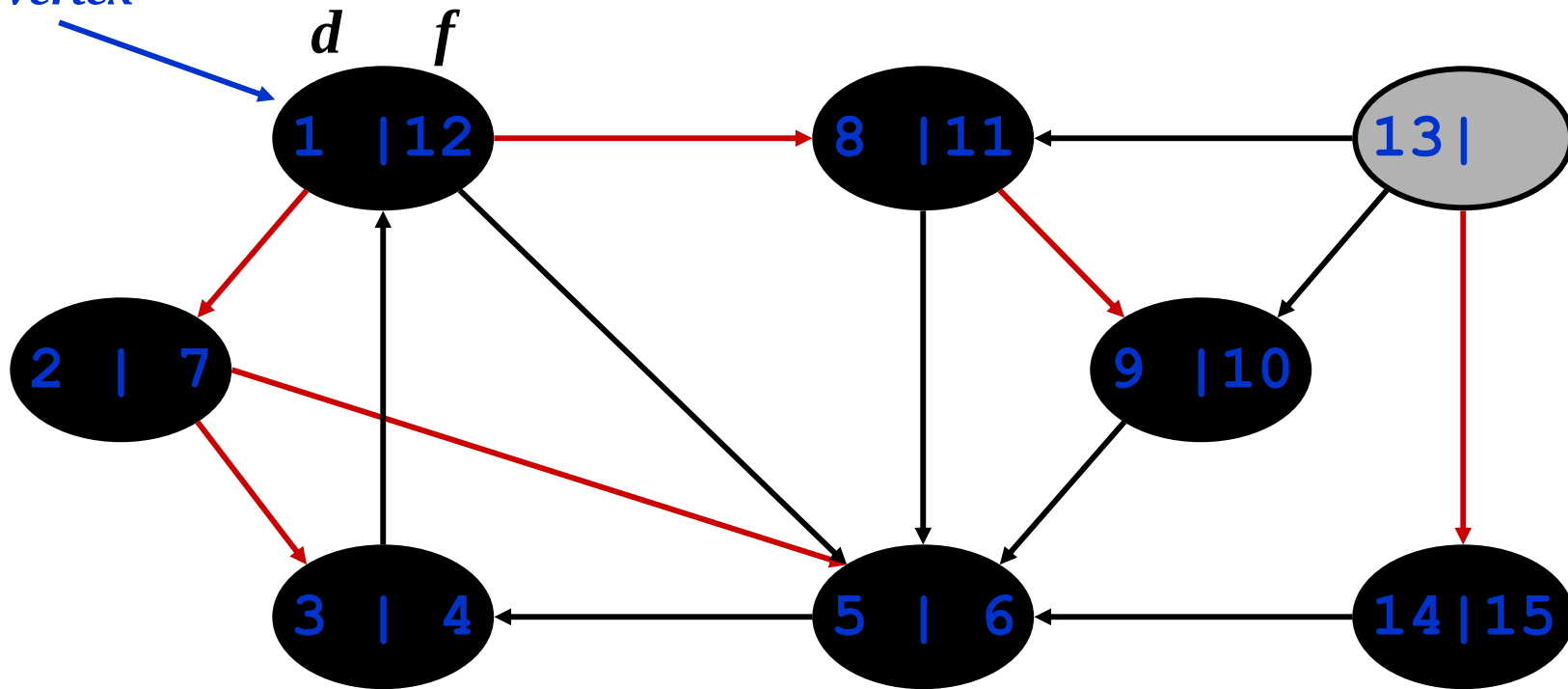
DFS Example

source
vertex



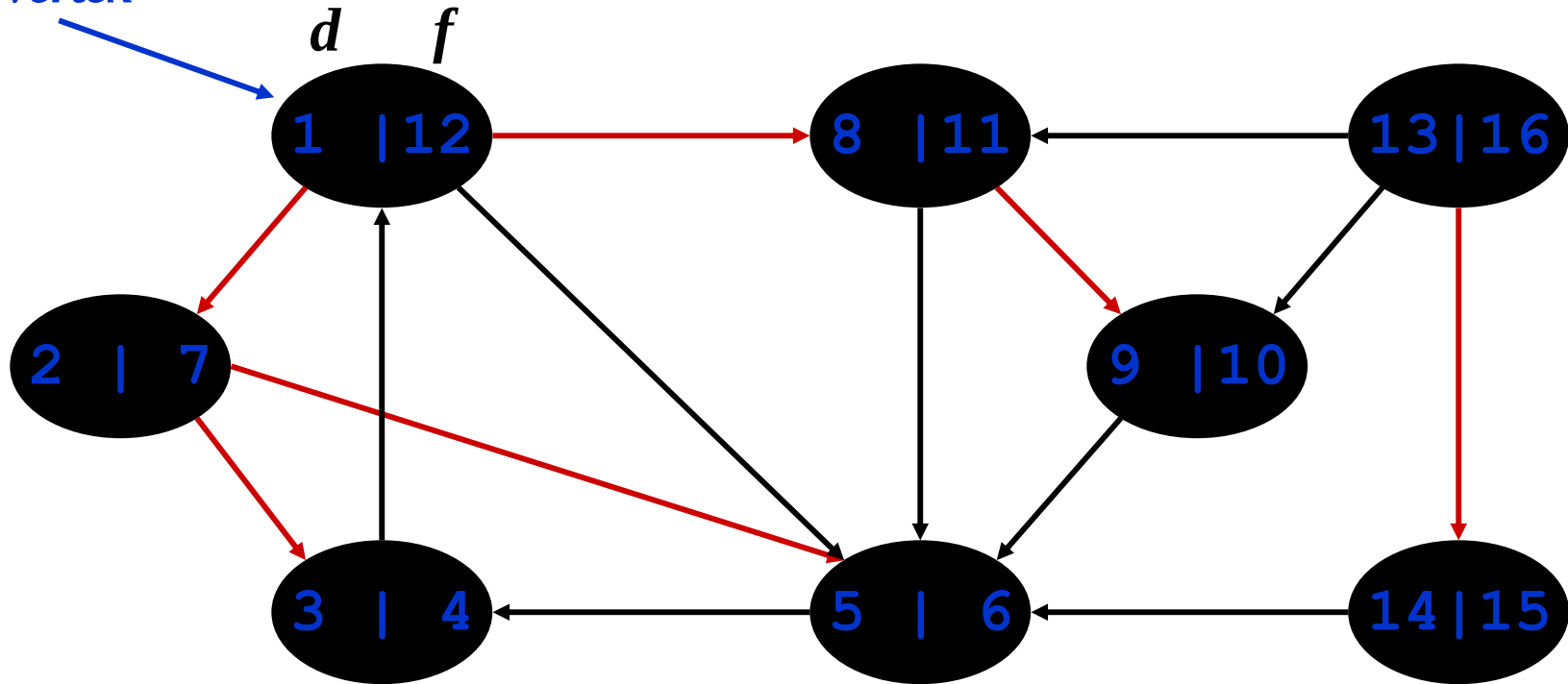
DFS Example

source
vertex



DFS Example

source
vertex

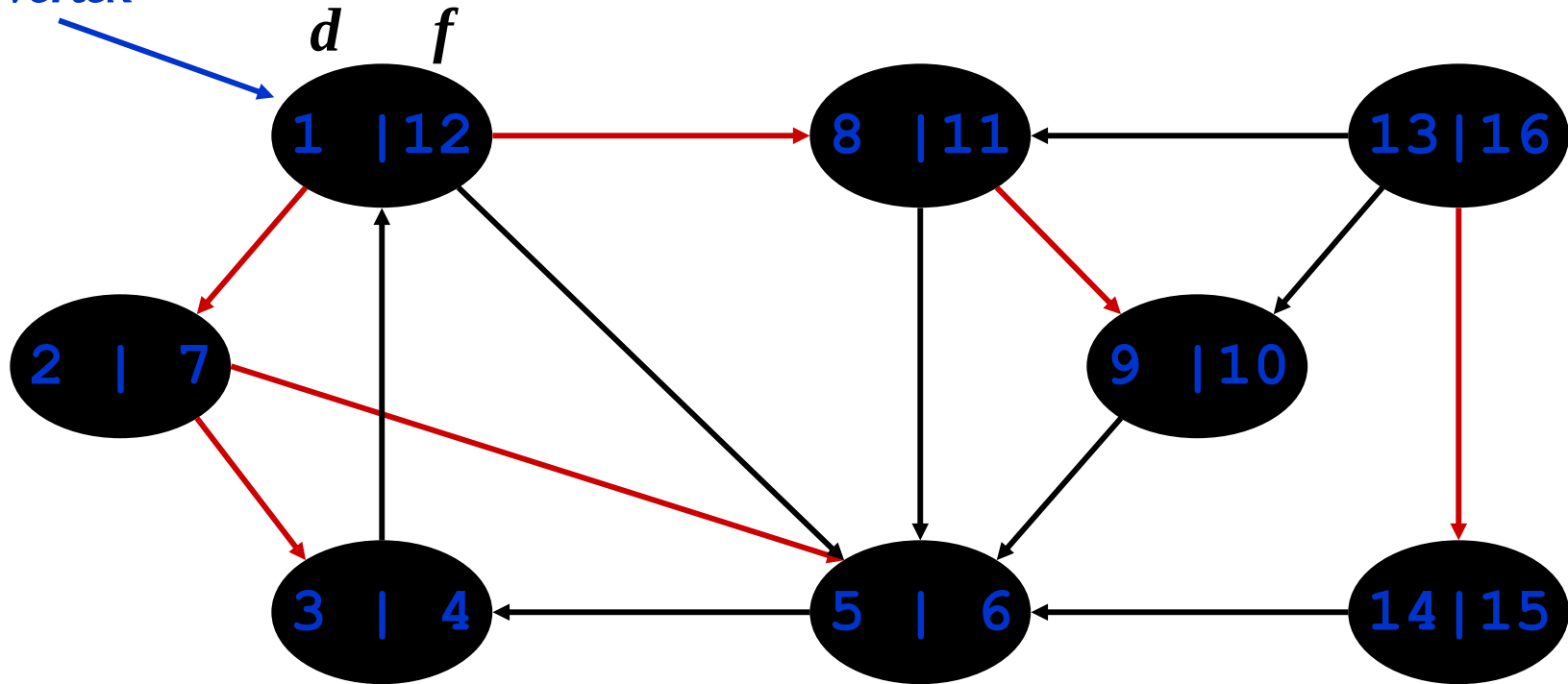


DFS: Kinds of edges

- DFS introduces an important distinction among edges in the original graph:
 - *Tree edge*: encounter new (white) vertex
 - The tree edges form a spanning forest
 - *Can tree edges form cycles? Why or why not?*

DFS Example

*source
vertex*



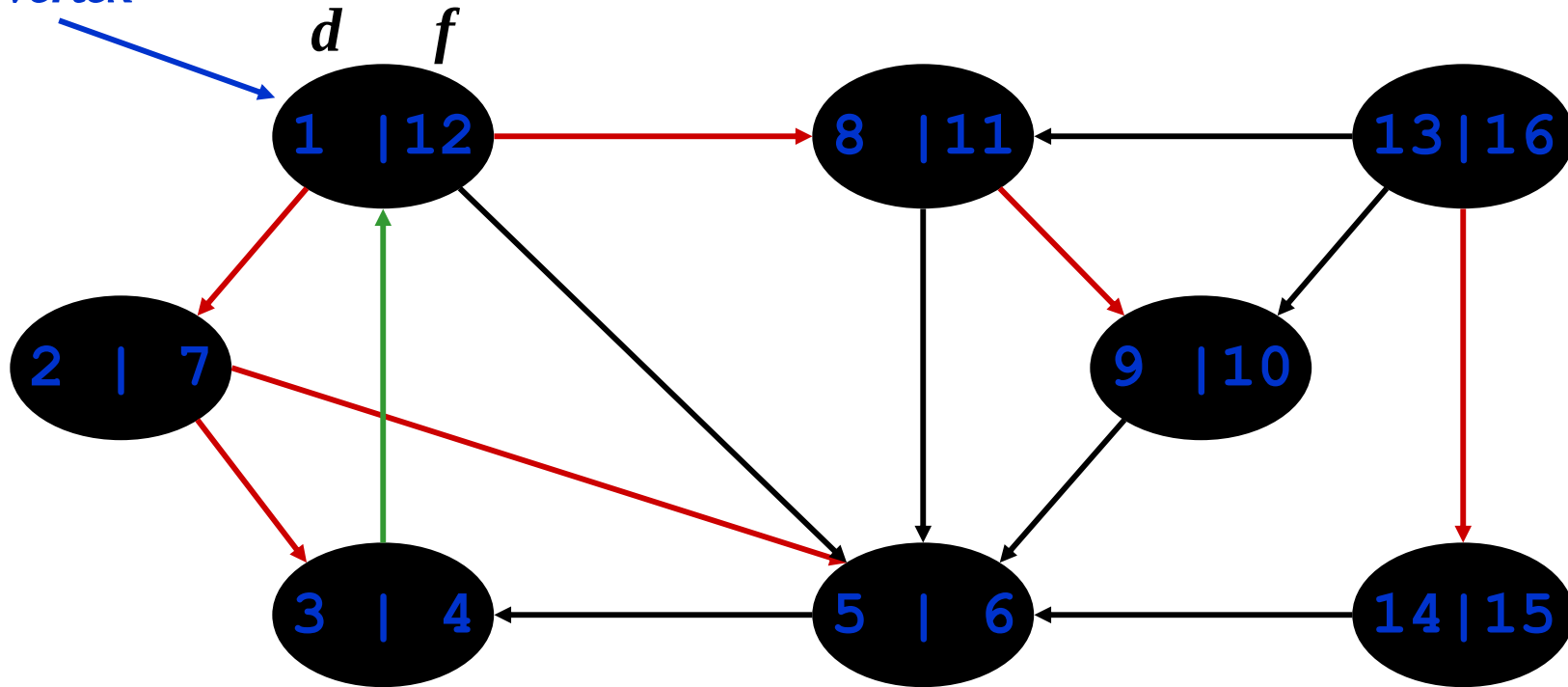
Tree edges

DFS: Kinds of edges

- DFS introduces an important distinction among edges in the original graph:
 - *Tree edge*: encounter new (white) vertex
 - *Back edge*: from descendent to ancestor
 - Encounter a grey vertex (grey to grey)

DFS Example

source
vertex



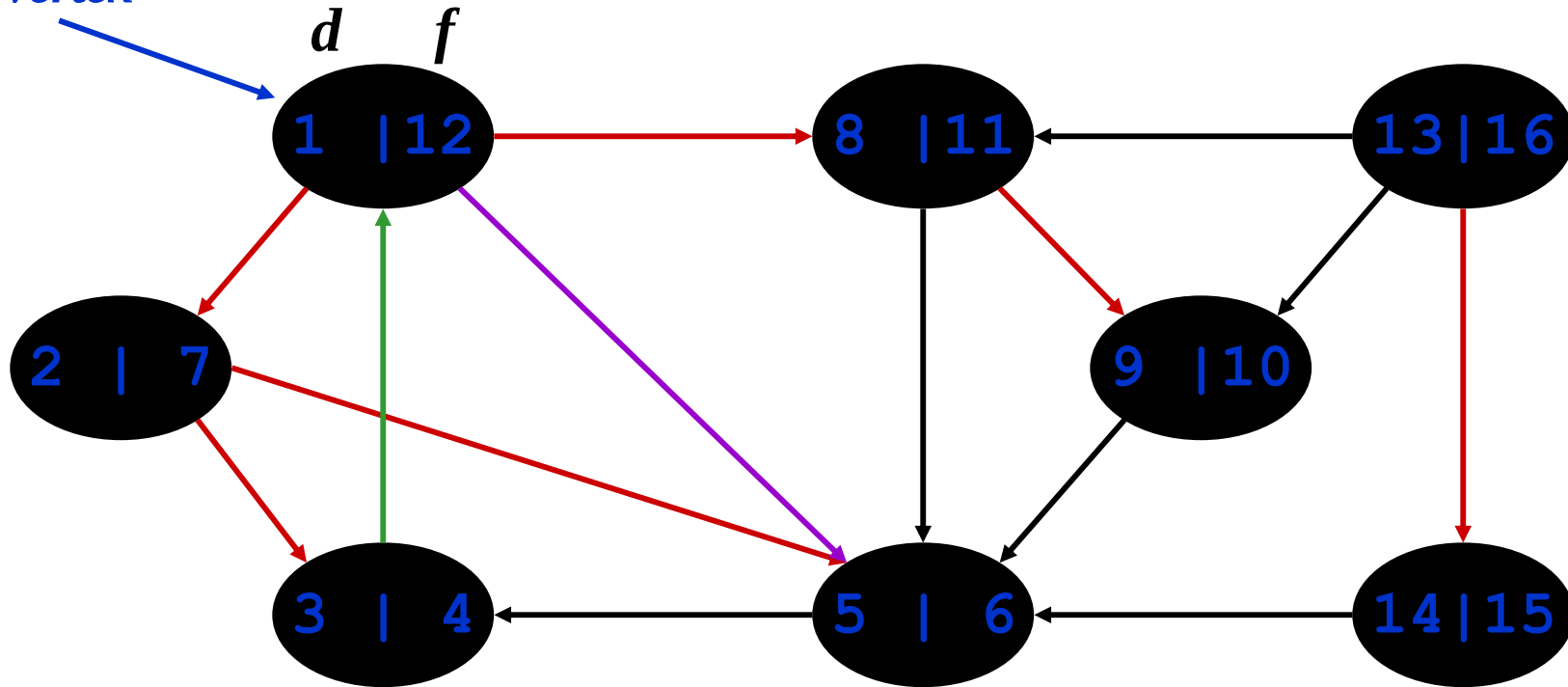
Tree edges *Back edges*

DFS: Kinds of edges

- DFS introduces an important distinction among edges in the original graph:
 - *Tree edge*: encounter new (white) vertex
 - *Back edge*: from descendent to ancestor
 - *Forward edge*: from ancestor to descendent
 - Not a tree edge, though
 - From grey node to black node

DFS Example

source
vertex



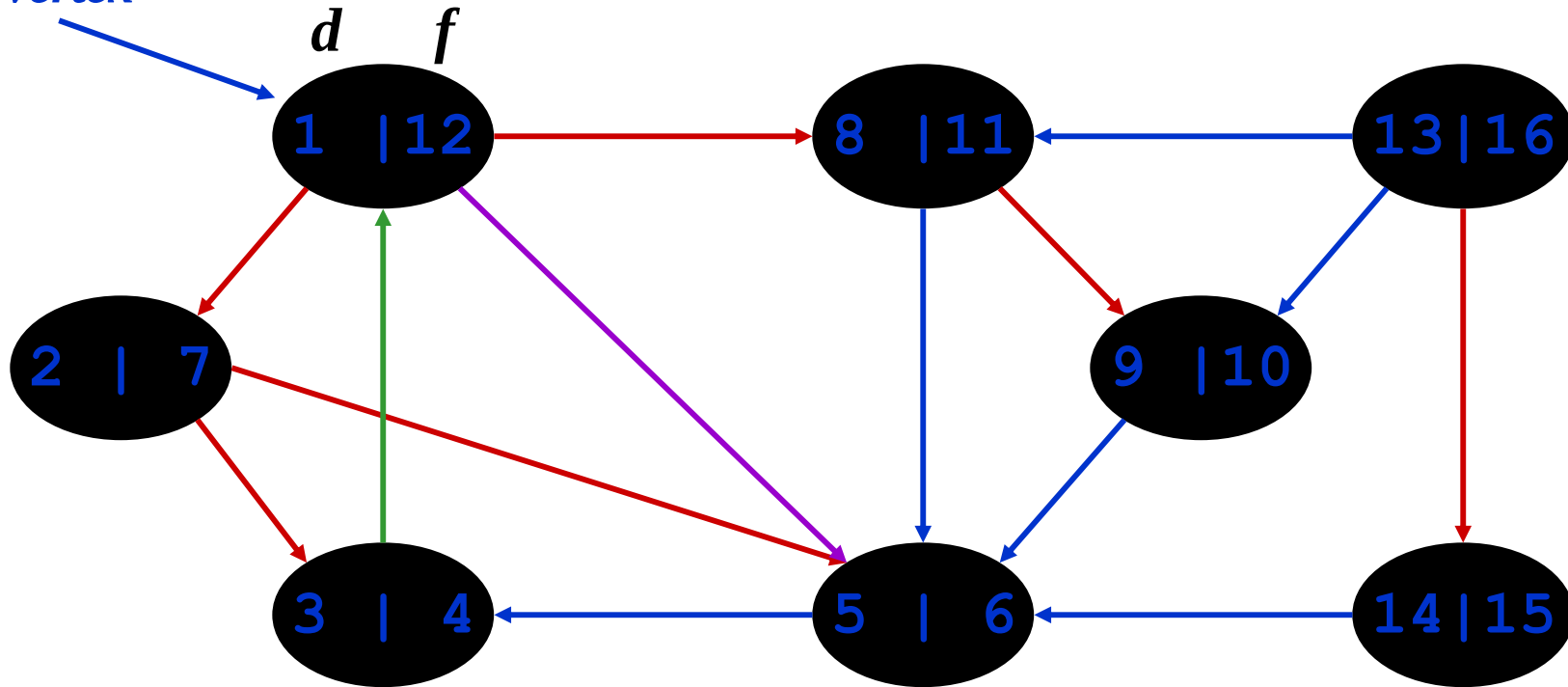
Tree edges *Back edges* *Forward edges*

DFS: Kinds of edges

- DFS introduces an important distinction among edges in the original graph:
 - *Tree edge*: encounter new (white) vertex
 - *Back edge*: from descendent to ancestor
 - *Forward edge*: from ancestor to descendent
 - *Cross edge*: between a tree or subtrees
 - From a grey node to a black node

DFS Example

source
vertex



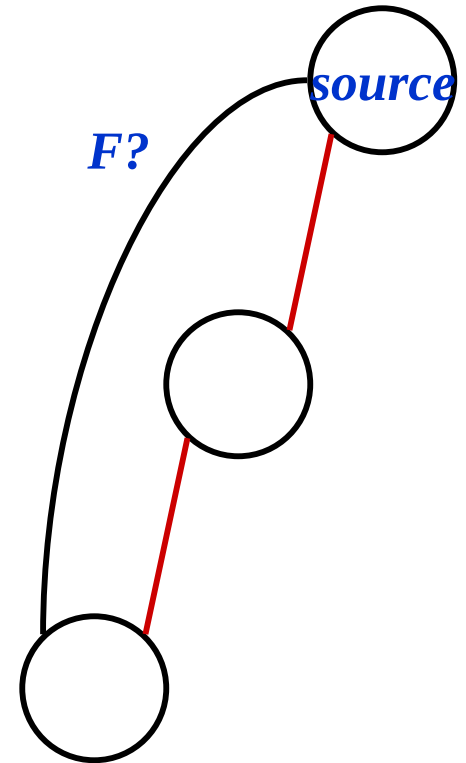
Tree edges *Back edges* *Forward edges* *Cross edges*

DFS: Kinds of edges

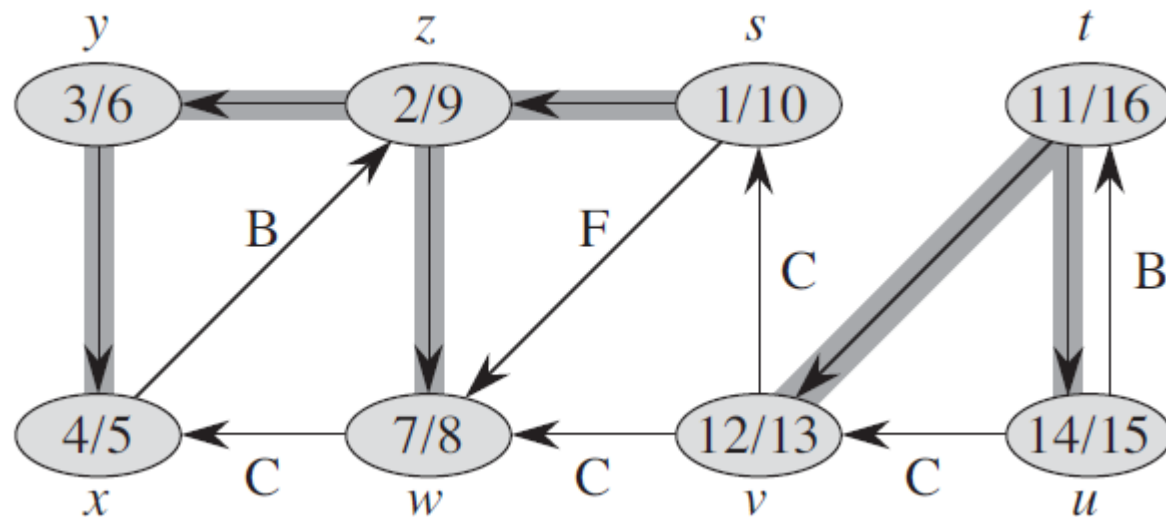
- DFS introduces an important distinction among edges in the original graph:
 - *Tree edge*: encounter new (white) vertex
 - *Back edge*: from descendent to ancestor
 - *Forward edge*: from ancestor to descendent
 - *Cross edge*: between a tree or subtrees
- Note: tree & back edges are important; most algorithms don't distinguish forward & cross

DFS: Kinds Of Edges

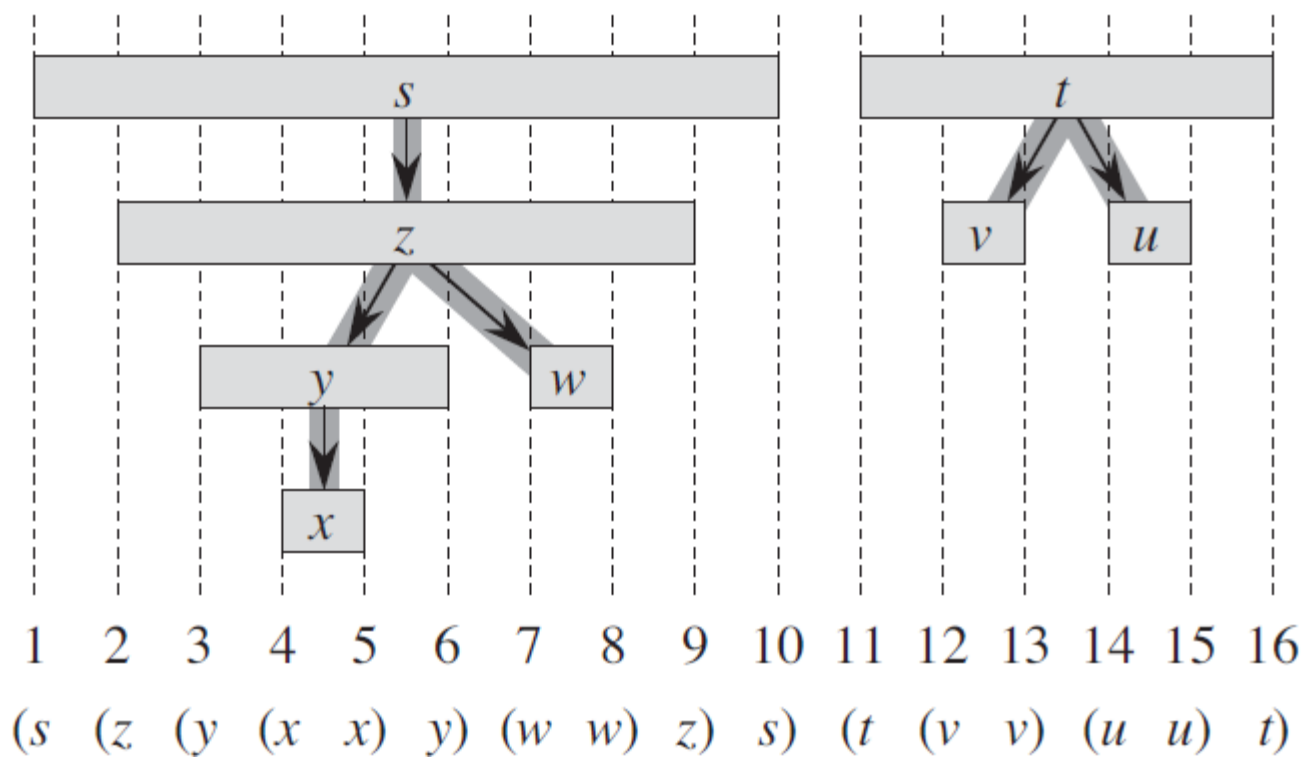
- Thm 23.9: If G is undirected, a DFS produces only tree and back edges
- Proof by contradiction:
 - Assume there's a forward edge
 - But $F?$ edge must actually be a back edge (*why?*)



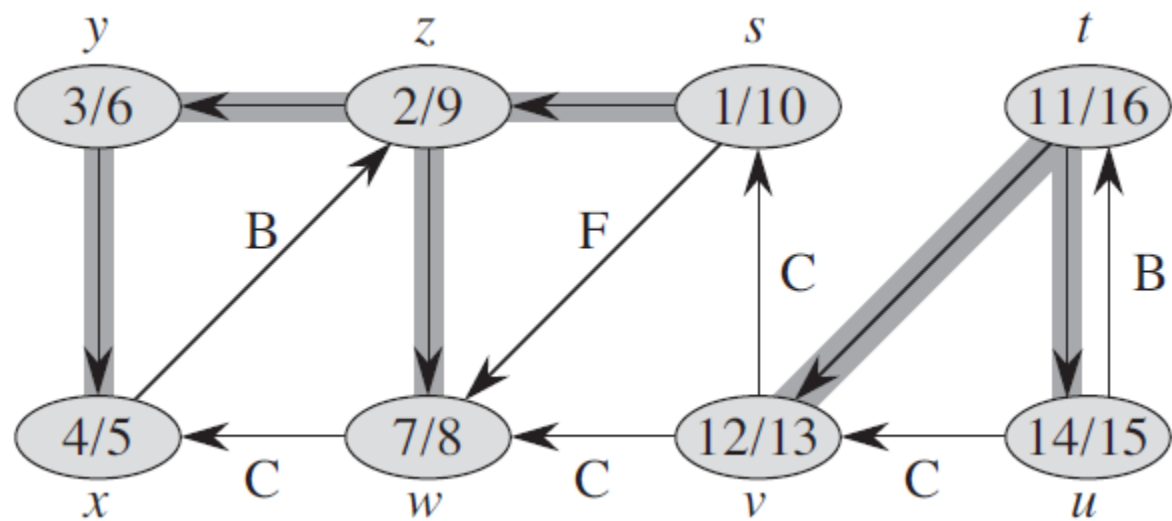
(a)



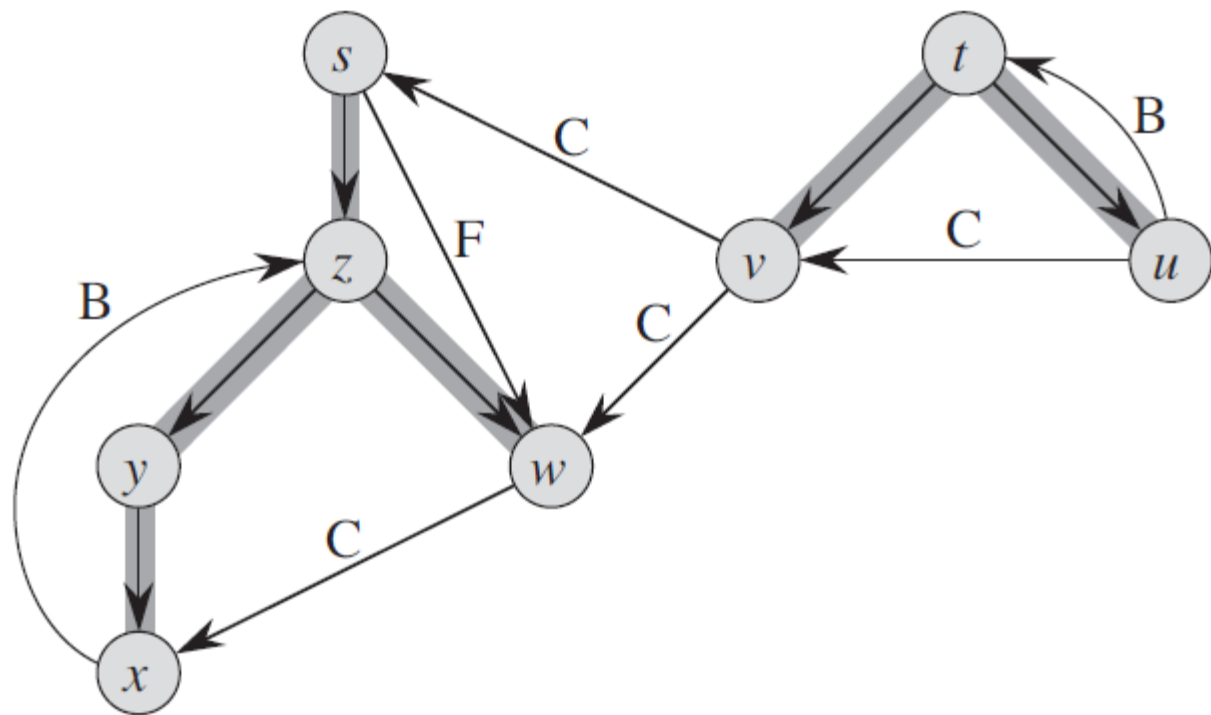
(b)



(a)



(c)



Theorem 22.7 (Parenthesis theorem)

In any depth-first search of a (directed or undirected) graph $G = (V, E)$, for any two vertices u and v , exactly one of the following three conditions holds:

- the intervals $[u.d, u.f]$ and $[v.d, v.f]$ are entirely disjoint, and neither u nor v is a descendant of the other in the depth-first forest,
- the interval $[u.d, u.f]$ is contained entirely within the interval $[v.d, v.f]$, and u is a descendant of v in a depth-first tree, or
- the interval $[v.d, v.f]$ is contained entirely within the interval $[u.d, u.f]$, and v is a descendant of u in a depth-first tree.

Corollary 22.8 (Nesting of descendants' intervals)

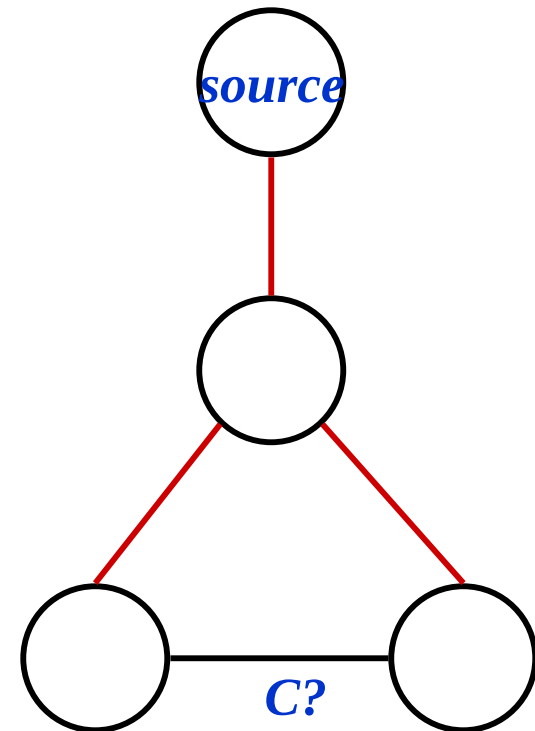
Vertex v is a proper descendant of vertex u in the depth-first forest for a (directed or undirected) graph G if and only if $u.d < v.d < v.f < u.f$.

Theorem 22.9 (White-path theorem)

In a depth-first forest of a (directed or undirected) graph $G = (V, E)$, vertex v is a descendant of vertex u if and only if at the time $u.d$ that the search discovers u , there is a path from u to v consisting entirely of white vertices.

DFS: Kinds Of Edges

- Thm 23.9: If G is undirected, a DFS produces only tree and back edges
- Proof by contradiction:
 - Assume there's a cross edge
 - But $C?$ edge cannot be cross:
 - must be explored from one of the vertices it connects, becoming a tree vertex, before other vertex is explored
 - So in fact the picture is wrong...both lower tree edges cannot in fact be tree edges



DFS And Graph Cycles

- Thm: An undirected graph is *acyclic* iff a DFS yields no back edges
 - If acyclic, no back edges (because a back edge implies a cycle)
 - If no back edges, acyclic
 - No back edges implies only tree edges (*Why?*)
 - Only tree edges implies we have a tree or a forest
 - Which by definition is acyclic
- Thus, can run DFS to find whether a graph has a cycle

DFS And Cycles

- *How would you modify the code to detect cycles?*

DFS (G)

```
{
    for each vertex u ∈ G->V
    {
        u->color = WHITE;
    }
    time = 0;
    for each vertex u ∈ G->V
    {
        if (u->color == WHITE)
            DFS_Visit(u);
    }
}
```

DFS_Visit(u)

```
{
    u->color = GREY;
    time = time+1;
    u->d = time;
    for each v ∈ u->Adj[]
    {
        if (v->color == WHITE)
            DFS_Visit(v);
    }
    u->color = BLACK;
    time = time+1;
    u->f = time;
}
```

DFS And Cycles

- *What will be the running time?*

DFS (G)

```
{
    for each vertex u ∈ G->V
    {
        u->color = WHITE;
    }
    time = 0;
    for each vertex u ∈ G->V
    {
        if (u->color == WHITE)
            DFS_Visit(u);
    }
}
```

DFS_Visit(u)

```
{
    u->color = GREY;
    time = time+1;
    u->d = time;
    for each v ∈ u->Adj[]
    {
        if (v->color == WHITE)
            DFS_Visit(v);
    }
    u->color = BLACK;
    time = time+1;
    u->f = time;
}
```

DFS And Cycles

- *What will be the running time?*
- A: $O(V+E)$
- We can actually determine if cycles exist in $O(V)$ time:
 - In an undirected acyclic forest, $|E| \leq |V| - 1$
 - So count the edges: if ever see $|V|$ distinct edges, must have seen a back edge along the way